

REPUBLIQUE DU CAMEROUN

Paix-Travail-Patrie

Ministère de l'Enseignement

Supérieur

Université de Douala

ECOLE SUPERIEURE DES
SCIENCES DE LA FINANCE, DE
L'ASSURANCE ET DES RISQUES

REPUBLIC OF CAMEROON

Peace-Work-Fatherland

Ministry of Higher Education

Douala University

HIGHER SCHOOL OF FINANCE,
INSURANCES AND RISK SCIENCES



THEME :

**LES BASES DE DONNEES VECTORIELLES ET LEUR
APPLICATIONS DANS L'ECOSYSTEME ORACLE : APPROCHE,
PERFORMANCES ET CAS D'USAGE.**

Rédigé par :

KOUNGA TCHOMNOU RYAN YVAN

Étudiant en Master 2 Statistiques, Big Data et Intelligence Artificielle

Encadreur Professionnel

Mr GABRIELE MOPOLO MOKE
Docteur en Informatique

ANNEE ACADEMIQUE
2024 – 2025

Résumé

Abstract

Liste des figures

Liste des acronymes

Plan du document

Table des matières

1.	Introduction générale (1 page)	6
2.	Présentation du projet	6
3.	Les concepts fondamentaux des bases de données vectorielles	7
3.1	Les bases de données vectorielles	7
	Définition	7
	Représentation des vecteurs (dense vs sparse)	7
	Recherche vectorielle : recherche de similarité et métriques	7
	Méthodes : brute force vs approximate (ANN).....	8
3.2	Architectures des systèmes vectoriels	8
3.3	Cas d'utilisation des technologies vectorielles.....	9
4.	État de l'art sur les moteurs de bases de données vectorielles	11
	4.1 Critères des études	11
	4.2. Oracle AI Vector	12
	4.3. pgvector	13
	4.4. Weaviate	14
	4.5. FAISS (Facebook AI Similarity Search)	15
	4.2 Tableau comparatif des moteurs de BD vectoriels	16
5.	Architecture et fonctionnement oracle 23ai et les base de données vectoriels	18
5.1	Objectifs	18
5.2	Composants principes d'oracle 23ai et les base de données vectorielles	19
	5.2.1 Architecture distribuée et résilience de oracle 23ai et les base de données vectoriels	19
	5.2.2 Cas d'usage concrets	19
5.3	Application de référence.....	20
6.	INSTALLATION d'ORACLE 23AI	22
6.1	Contexte et enjeux.....	22
6.2	Objectifs de cette première phase	22
6.3	Pré-requis matériels et logiciels	22
	6.3.1 Environnement matériel utilisé	22
	6.3.2 Choix du system d'exploitations	23
	6.3.3 Téléchargement.....	23
	6.3.4 Validation des prérequis.....	23
	6.3.5 Outils complémentaires.....	23
	6.3.6 Bilan de la phase préparatoire	24
6.4	Choix de la version : Oracle Database 23ai	24
	6.4.2 Téléchargement de l'archive Oracle	24
	6.4.3 Extraction et lancement de l'installation.....	24
	6.4.4 Préparation avant exécution	25

6.4.5 Lancement de l'installation.....	25
6.4.6 Bilan de l'étape	25
6.5 Installation et post-installation	26
6.5.1 Démarrage de l'installation.....	26
6.5.2 Structure des dossiers et chemins d'installation.....	26
6.5.3 Services Oracle installés	26
6.5.4 Premier test de connexion avec SQL*Plus	27
6.5.5 Vérification du listener Oracle	27
6.5.6 Connexion à la base PDB.....	28
6.5.7 Résultat de l'installation	29
7. MANIPULATION ET PRISE EN MAIN D'ORACLE 23ai.....	30
7.1. Manipulations de base : création d'utilisateurs, tables, insertion de données et requêtes SQL.....	30
7.1.1 Création d'un utilisateur Oracle personnalisé	30
7.1.2 Connexion avec l'utilisateur créé.....	30
7.1.3 Création d'une table simple	31
7.1.4 Insertion de données.....	31
7.1.5 Requetes SQL de lecture (SELECT).....	31
7.1.6 Mise à jour et suppression de données	32
7.1.7 Synthèse	32
7.2. Prise en main des bases de données vectorielles avec Oracle 23ai	33
7.2.1 Contexte et motivation.....	33
7.2.2 Recherche vectorielle intégrée	33
7.2.3 Création d'une table plus avancée : vecteurs NLP.....	33
7.2.3 Intégration avec des modèles IA et NLP	33
7.2.4 Insertion des données vectorielles en Python.....	34
7.2.5 Requetes SQL sur la table (en Python)	34
7.2.6 Résumé.....	35
7.2.7 Performances et observations	36
7.2.8 Synthèse	36
7.3 Conclusion de la phase d'installation et de prise en main.....	37
8. Intégration d'oracle 23ai et les base de données vectoriels dans un environnement Bigdata/data lake	38
8.1 Introduction.....	38
8.2.1 Sources de données	38
8.2.2 Intégration via MySQL et Apache Hop	39
8.2.3 Passerelle ODBC entre Oracle et MySQL	40
8.2.4 Stockage centralisé dans Oracle 23AI.....	41
8.3 Oracle 23AI comme moteur vectoriel dans le Data Lake	41
8.3.1 Génération des vecteurs	41

8.3.2 Stockage des vecteurs dans Oracle	44
8.4 Analyses avancées dans le Data Lake	45
8.4.1 Analyses PCA et DBSCAN sur le texte et les images des données médicales	45
8.4.2 Recherche de Similarité Vectorielle sur les image et texte en SQL avec Oracle AI	49
8.4.3 Health Care data Analyses avec python	49
8.5 Apport de l'intégration Oracle 23AI dans un Data Lake	49
8.6 Conclusion	50
9. Conclusion générale et perspectives	50
10. Bibliographie	50
Chapitre 3 – Concepts fondamentaux des bases de données vectorielles	50
Chapitre 4 – État de l'art et critères comparatifs.....	50
Fondements théoriques et architecture.....	50
Chapitre 5 – Oracle 23ai et BD vectorielles.....	51
Chapitre 6 – Intégration de ORACLE 23AI ET LES BD VECTORIELS dans un environnement Bigdata/data lake	51

1. Introduction générale (1 page)

Accroche avec un fait marquant, une stat, un slogan pub (bonus pour attirer l'attention dès le départ)

- > Les raisons qui vous ont poussé à choisir cette étude
- > Vos objectifs /attentes vis à vis de l'étude
- > Un bref descriptif de vos missions pendant l'étude
- > La problématique de votre rapport
- > L'annonce du plan de votre rapport de l'étude (présenter sur une ligne chaque chapitre)

2. Présentation du projet

<Voir le sujet de votre projet>

3. Les concepts fondamentaux des bases de données vectorielles

3.1 Les bases de données vectorielles

Définition

Les bases de données vectorielles constituent une nouvelle génération de systèmes de gestion de données conçus pour stocker, indexer et interroger des représentations numériques de données non structurées (texte, image, audio, vidéo). Contrairement aux bases relationnelles traditionnelles, qui manipulent des données tabulaires, les bases vectorielles travaillent avec des vecteurs, c'est-à-dire des représentations mathématiques en haute dimension produites par des modèles d'apprentissage automatique. Ces vecteurs capturent la sémantique ou les caractéristiques essentielles d'un objet (par exemple, le sens d'une phrase, les traits visuels d'une image ou la signature audio d'un signal).

Représentation des vecteurs (dense vs sparse)

Deux grandes formes de représentations coexistent :

- **Vecteurs denses** : ils sont généralement produits par des modèles de type *deep Learning* (par exemple, les *transformer* ou les réseaux convolutifs). Chaque dimension contient une valeur réelle, et l'ensemble des dimensions encode des informations compactes sur le contenu. Ces représentations sont largement utilisées pour la recherche sémantique, la reconnaissance d'images ou encore la recommandation.
- **Vecteurs clairsemés (sparse)** : ils proviennent souvent de méthodes classiques comme le *bag-of-Word* ou la pondération *TF-IDF*. Les vecteurs sont de grande dimension, mais contiennent majoritairement des zéros. Bien que plus simples, ils sont moins efficaces pour capturer la sémantique profonde des données

Recherche vectorielle : recherche de similarité et métriques

La recherche vectorielle consiste à retrouver, pour un vecteur donné (représentant une requête), les vecteurs les plus proches dans l'espace multidimensionnel. Cette proximité est mesurée par une **fonction de distance** ou **métrique de similarité**. Les principales métriques utilisées sont :

1. Distance Euclidienne (L2 Norm)

La **distance euclidienne** est l'une des métriques les plus utilisées.

Pour deux vecteurs $x = (x_1, x_2, \dots, x_n)$ et $y = (y_1, y_2, \dots, y_n)$:

$$d_{\{euclidienne\}(x,y)} = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

2. Distance de Manhattan (L1 Norm)

Elle calcule la somme des différences absolues entre les coordonnées :

$$d_{\{euclidienne\}}(x,y) = \sum_{i=1}^n |x_i - y_i|$$

3. Distance Cosinus (Cosine Similarity)

Très utilisée en **NLP** et pour la recherche sémantique, car elle mesure l'**angle** entre deux vecteurs (indépendamment de leur norme) :

$$sim_{\{cos\}}(x,y) = \frac{x \cdot y}{||x|| \cdot ||y||} = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \cdot \sqrt{\sum_{i=1}^n y_i^2}}$$

La distance cosinus peut être dérivée comme :

$$d_{\{cos\}}(x,y) = 1 - sim_{\{cos\}}(x,y)$$

Ces métriques déterminent la qualité et la pertinence des résultats lors des recherches de similarité

Méthodes : brute force vs approximate (ANN)

Deux grandes approches se distinguent :

- **Brute Force (exhaustive search)** : consiste à comparer chaque vecteur avec l'ensemble de la base. Bien que simple et garantissant des résultats exacts, cette méthode devient coûteuse en temps de calcul dès que la base atteint des millions de vecteurs.
- **Approximate Nearest Neighbor (ANN)** : repose sur des algorithmes d'indexation avancés qui accélèrent la recherche en sacrifiant légèrement la précision. Parmi les techniques les plus utilisées, on retrouve :
 - **HNSW (Hierarchical Navigable Small World graphs)**, efficace pour de grandes bases.
 - **IVF (Inverted File Index)**, souvent couplé à une quantification (PQ, OPQ).
 - **LSH (Locality-Sensitive Hashing)**, basé sur des fonctions de hachage adaptées aux distances.

Ces méthodes permettent d'obtenir des recherches en temps quasi réel, même sur des bases massives.

3.2 Architectures des systèmes vectoriels

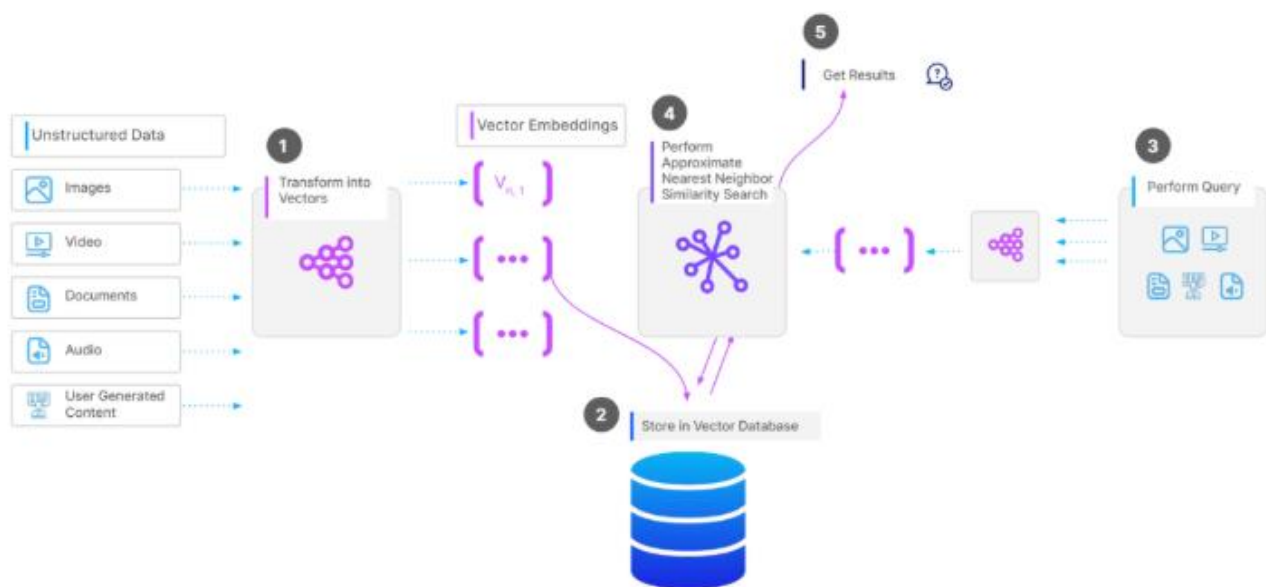


Figure: Complete workflow of a vector search operation

Les systèmes vectoriels adoptent généralement une **architecture distribuée** et optimisée pour les requêtes de similarité. Il combine généralement :

- **Stockage spécialisé** : basé sur des moteurs optimisés pour gérer des vecteurs de haute dimension. Certains systèmes s'appuient sur des bases relationnelles étendues (ex. : PostgreSQL avec pgvector), tandis que d'autres reposent sur des moteurs natifs (Weaviate, Pinecone).
- **Indexation avancée** : intégration de structures ANN (HNSW, IVF, etc.) pour accélérer la recherche.
- **Parallélisation et distribution** : afin de gérer des volumes massifs de données, plusieurs bases vectorielles adoptent une architecture distribuée avec *sharding* et réplication.
- **Intégration IA-native** : certains moteurs incluent directement des connecteurs avec des modèles d'apprentissage automatique (ex. Oracle AI Vector Search intégré à Oracle Database 23ai).

3.3 Cas d'utilisation des technologies vectorielles

Traitement automatique du langage naturel (NLP)

Les vecteurs sont essentiels dans des tâches comme :

- **Recherche sémantique** : retrouver un document pertinent en fonction d'une requête en langage naturel.
- **Question Answering (QA)** : associer une question utilisateur à la réponse la plus proche dans une base documentaire.
- **Chatbots avancés** : utiliser des vecteurs pour enrichir la mémoire contextuelle et améliorer la pertinence des réponses.

Vision par ordinateur

Dans le domaine de l'analyse d'images et de vidéos :

- **Recherche d'images par similarité** : retrouver des images proches d'une requête donnée (par exemple, trouver des produits similaires dans un catalogue e-commerce).
- **Détection et classification d'objets** : utiliser des vecteurs issus de réseaux convolutifs pour indexer et retrouver des objets visuels.
- **Analyse vidéo** : segmenter et retrouver des séquences pertinentes dans des contenus multimédias.

Systèmes de recommandation

- **Recommandations personnalisées** : comparer le profil vectoriel d'un utilisateur avec celui d'items (films, produits, musiques).
- **Détection d'anomalies** : identifier des comportements atypiques en mesurant leur éloignement dans l'espace vectoriel.
- **Publicité ciblée** : associer vecteurs de préférences utilisateur et vecteurs de campagnes publicitaires.

4. État de l'art sur les moteurs de bases de données vectorielles

L'essor massif des données non structurées (textes, images, vidéos, signaux audio) conjugué aux avancées des modèles d'intelligence artificielle (IA) a transformé les méthodes de stockage et d'interrogation de l'information. Les bases de données vectorielles, ou moteurs de recherche vectorielle, se sont imposées comme une réponse incontournable à ce besoin. Leur rôle principal consiste à représenter les données sous forme de vecteurs numériques (embeddings), à les indexer efficacement et à permettre des recherches par similarité sémantique.

Contrairement aux bases relationnelles classiques, qui gèrent essentiellement des données structurées, les moteurs de bases de données vectorielles exploitent des algorithmes avancés d'indexation et de recherche approximative (ANN – Approximate Nearest Neighbors). Ces approches garantissent une exploration rapide et scalable de milliards de vecteurs tout en préservant une qualité de correspondance élevée.

Dans ce contexte, plusieurs solutions ont émergé, portées par de grands acteurs de l'industrie et de la recherche, chacune proposant une combinaison spécifique d'architectures, de performances et de fonctionnalités. Cette section propose un état de l'art comparatif de moteurs emblématiques, à savoir **Oracle AI Vector**, **pgvector**, **Weaviate** et **FAISS**, en s'appuyant sur des critères rigoureusement définis.

4.1 Critères des études

Afin d'évaluer et de comparer les moteurs de bases de données vectorielles, plusieurs critères pertinents ont été retenus :

1. Fondements théoriques et architecture

Ce critère examine les concepts sous-jacents, le modèle de stockage adopté (natif ou extension), ainsi que l'architecture logicielle (centralisée, distribuée, cloud-native).

2. Indexation et recherche vectorielle

Analyse des mécanismes d'indexation proposés (HNSW, IVF, PQ, Flat, etc.), de la gestion de la recherche exacte vs approximative, ainsi que des stratégies d'optimisation.

3. Performance

Mesure de la scalabilité, du temps de réponse, de la capacité à gérer de grands volumes de vecteurs et de la consommation de ressources.

4. Fonctionnalités avancées

Évaluation des services complémentaires : support multimodal (texte, image, audio), gestion des métadonnées, intégration avec des modèles de machine learning, fonctionnalités analytiques et support du temps réel.

5. Interopérabilité et intégration

Capacité du moteur à s'intégrer dans des pipelines de données existants, compatibilité avec d'autres bases de données ou frameworks IA, support d'API et connecteurs.

6. Cas d'usage

Illustration des principaux domaines d'application (recherche sémantique, recommandation, détection de fraude, vision par ordinateur, assistants IA, etc.).

7. Algorithmes de recherche de similarité

Description des méthodes employées pour mesurer la proximité entre vecteurs (cosinus, L2, dot product, etc.) et prise en charge de métriques multiples.

8. Aspects pratiques

Prise en compte de la facilité d'installation, documentation, communauté d'utilisateurs, support commercial éventuel et coût d'adoption (open source vs propriétaire).

4.2. Oracle AI Vector

Fondements théoriques et architecture

Oracle AI Vector est une extension native intégrée à Oracle Database 23ai. Elle repose sur l'architecture **multimodèle** d'Oracle, permettant de gérer simultanément données relationnelles, JSON, spatiales et désormais vectorielles. Contrairement à certains moteurs spécialisés, Oracle ne crée pas une base dédiée mais enrichit sa base relationnelle avec le support des vecteurs via le type VECTOR.

Indexation et recherche vectorielle

Oracle propose des **index vectoriels basés sur HNSW (Hierarchical Navigable Small World)** pour la recherche approximative de similarité (ANN). Les vecteurs sont stockés et indexés comme des colonnes natives, ce qui permet des requêtes SQL classiques enrichies d'opérateurs tels que COSINE_DISTANCE, DOT_PRODUCT ou EUCLIDEAN_DISTANCE.

Performance

Les performances sont optimisées grâce à l'intégration directe dans le moteur Oracle, ce qui évite des appels externes. Oracle exploite aussi ses fonctionnalités de parallélisation et de partitionnement pour accélérer les requêtes sur de très grands volumes de données.

Fonctionnalités avancées

- Recherche hybride (combiner filtres relationnels + recherche vectorielle).
- Intégration avec **Oracle Machine Learning** et des modèles externes.
- Sécurité avancée (Data Masking, Audit, RBAC) héritée d'Oracle.
- Support pour **RAG (Retrieval Augmented Generation)** avec LLMs.

Interopérabilité et intégration

Compatible avec tout l'écosystème Oracle (APEX, GoldenGate, Data Pump, Big Data SQL). Oracle AI Vector se connecte aussi à des pipelines IA via **Python, Java, REST API** et supporte OCI AI Services.

Cas d'usage

- Recherche multimodale (texte, image, audio).
- Génération augmentée par la recherche (chatbots).
- Intégration dans les entrepôts de données d'entreprise.

Algorithmes de recherche de similarité

Support natif de la **distance cosinus**, de la **distance euclidienne** et du **produit scalaire**.

Aspects pratiques

L'installation se fait automatiquement avec Oracle 23ai. Aucune infrastructure externe n'est nécessaire. Le coût reste lié au modèle de licence Oracle, ce qui peut être un frein pour des usages académiques ou petites entreprises.

4.3. pgvector

Fondements théoriques et architecture

pgvector est une extension de **PostgreSQL** ajoutant un type de données vector. Elle repose sur la philosophie open source et l'architecture client-serveur PostgreSQL.

Indexation et recherche vectorielle

Supporte plusieurs index :

- **IVFFlat (Inverted File Index)** pour la recherche ANN.
- **HNSW** depuis les versions récentes.
- Scans séquentiels pour de petits jeux de données.

Performance

Moins performant que des moteurs spécialisés comme FAISS, mais suffisant pour des ensembles de taille moyenne (< 100M vecteurs). La performance dépend fortement du tuning de PostgreSQL (indexation, mémoire).

Fonctionnalités avancées

- Intégration SQL native.

- Support des distances : **cosine**, **L2 (euclidean)**, **inner product**.
- Peut être combiné avec toutes les extensions PostgreSQL (PostGIS, TimescaleDB, etc.).

Interopérabilité et intégration

Très flexible car s'intègre dans tout environnement basé sur PostgreSQL. Compatible avec **LangChain**, **LlamaIndex**, et la plupart des frameworks IA.

Cas d'usage

- Recherche sémantique dans des applications web.
- Stockage vectoriel léger pour projets IA open source.
- Prototypage rapide avec LLMs.

Algorithmes de recherche de similarité

Support : **cosinus**, **euclidienne**, **inner product**.

Aspects pratiques

Installation simple (CREATE EXTENSION pgvector). Open source et gratuit, avec une large communauté. Moins adapté à l'échelle industrielle très massive (>1 milliard de vecteurs).

4.4. Weaviate

Fondements théoriques et architecture

Weaviate est une base de données vectorielle **native et open source** conçue spécifiquement pour les applications IA. Elle repose sur une architecture distribuée orientée microservices.

Indexation et recherche vectorielle

Utilise principalement **HNSW** pour l'ANN. Permet une recherche hybride combinant **BM25 (recherche full-text)** et vecteurs.

Performance

Optimisé pour la scalabilité horizontale. Peut gérer des milliards de vecteurs avec une recherche rapide grâce à l'index HNSW distribué.

Fonctionnalités avancées

- API GraphQL pour la recherche sémantique.
- Support des multimodaux (texte, image, audio, vidéo).
- Intégration directe de modèles de ML pour la vectorisation (OpenAI, HuggingFace).

- Support du **RAG** et de pipelines IA.

Interopérabilité et intégration

Facilement intégrable avec **LangChain**, **Transformers**, **OpenAI API**.

Propose un **hébergement managé (Weaviate Cloud Service)** ou installation on-premise.

Cas d'usage

- Chatbots IA en production.
- Recherche multimodale sur de grands corpus.
- Applications temps réel (par ex. moteurs de recommandation).

Algorithmes de recherche de similarité

Supporte **cosinus**, **euclidienne**, **produit scalaire**.

Aspects pratiques

Solution clé en main avec API simple. Requiert cependant une infrastructure Kubernetes ou Docker pour un déploiement distribué.

4.5. FAISS (Facebook AI Similarity Search)

Fondements théoriques et architecture

FAISS est une **bibliothèque C++/Python développée par Meta AI**, optimisée pour la recherche vectorielle et le clustering. Contrairement aux autres, ce n'est pas une base de données complète mais une librairie spécialisée.

Indexation et recherche vectorielle

Propose de nombreux index ANN :

- **IVF (Inverted File Index)**
- **PQ (Product Quantization)**
- **HNSW**
- **Flat (brute force)**

Performance

Extrêmement performant et optimisé GPU. Référence académique et industrielle pour l'ANN sur très grands volumes (>1 milliard vecteurs).

Fonctionnalités avancées

- Compression vectorielle pour économiser mémoire et stockage.
- Recherche GPU pour accélération massive.
- Outils de clustering (k-means).

Interopérabilité et intégration

S'intègre via **Python** avec NumPy, PyTorch, HuggingFace. Peut être combiné avec une base relationnelle pour le stockage des métadonnées.

Cas d'usage

- Systèmes de recommandation à grande échelle.
- Recherche multimédia (images, vidéos).
- Expérimentation IA à grande échelle.

Algorithmes de recherche de similarité

Support d'une large gamme : cosinus, L2, produit scalaire.

Aspects pratiques

Très performant mais nécessite une gestion externe des métadonnées (pas une base clé-valeur complète). Installation simple via `pip install faiss-cpu` ou `faiss-gpu`.

4.2 Tableau comparatif des moteurs de BD vectoriels

Critères	Oracle AI Vector	pgvector	Weaviate	FAISS
Fondements théoriques et architecture	Intégré nativement à Oracle 23ai ; architecture relationnelle + vecteurs dans des colonnes dédiées ; support multi-modèle.	Extension PostgreSQL (open source) ; ajoute un type <code>vector</code> et des fonctions associées.	Base de données vectorielle cloud-native ; API REST/GraphQL ; pensée pour la recherche sémantique.	Librairie C++/Python développée par Meta ; pas une BD complète, mais un moteur spécialisé dans la recherche vectorielle.
Indexation et recherche vectorielle	Index vectoriels (HNSW, IVF, brute force) intégrés dans Oracle ; support des distances Cosine, Euclidienne, Inner Product.	Index IVFFlat, HNSW ; distances Euclidienne, Cosine, Inner Product.	Index HNSW optimisé pour la recherche sémantique ; support de filtres hybrides (mots-clés + vecteurs).	Index puissants (Flat, IVFFlat, IVFPQ, HNSW) ; spécialisé dans l'approximate nearest neighbor (ANN).

Performance	Haute performance grâce à l'intégration Oracle et au parallélisme ; adapté aux très grands volumes (Big Data).	Performances correctes mais dépendantes des ressources PostgreSQL ; efficace pour des bases de taille moyenne.	Optimisé pour scalabilité horizontale et cloud ; très rapide sur données non structurées.	Très haute performance sur millions à milliards de vecteurs ; optimisé GPU et CPU.
Fonctionnalités avancées	Combinaison SQL + vector search ; intégration avec données relationnelles, JSON, multimodèle (texte, image, vidéo).	Simple et léger ; peu de fonctionnalités avancées hors PostgreSQL standard.	Recherche multimodale (texte, image, audio) ; intégration NLP ; génération de schémas automatiques.	Support quantisation, compression, recherche ANN massive ; mais pas de gestion de transactions ni de données relationnelles.
Interopérabilité et intégration	Intégré à l'écosystème Oracle (PL/SQL, Big Data SQL, OCI) ; interfaçage Python, Java, etc.	Fonctionne dans l'écosystème PostgreSQL ; interfaçage Python, R, etc.	API REST, GraphQL, clients pour Python, Go, Java ; intégration avec modèles NLP (Hugging Face, OpenAI).	Disponible en C++/Python ; utilisé dans d'autres BD vectorielles (Milvus, Weaviate).
Cas d'usage	Recherche multimodale (documents, images, vidéos) ; finance, santé, e-commerce ; intégration dans systèmes critiques.	Recherche sémantique et recommandation ; petites/moyennes entreprises ; intégration rapide dans applications PostgreSQL.	Recherche sémantique dans le cloud ; moteurs de recherche contextuels ; assistants IA multimodaux.	Recherche à grande échelle (images, vidéos, embeddings) ; utilisé en recherche académique et industrielle (reco, NLP, vision).
Algorithmes de similarité	Cosine, Euclidienne (L2), Inner Product.	Cosine, Euclidienne, Inner Product.	Principalement Cosine et L2 ; via HNSW.	Large choix (L2, Cosine, IP) + variantes optimisées ANN.

Au terme de cette étude comparative des principaux moteurs de bases de données vectorielles, il apparaît que chaque solution possède des avantages spécifiques :

- Les moteurs spécialisés comme **Pinecone** ou **Weaviate** offrent une grande flexibilité et des performances élevées dans des environnements cloud natifs.
- Des solutions open source comme **Milvus** ou **FAISS** se distinguent par leur rapidité et leur adaptabilité, mais exigent une expertise technique importante pour l'intégration et la maintenance.
- Les SGBD traditionnels enrichis de fonctionnalités vectorielles (PostgreSQL + pgvector, MongoDB Atlas Vector Search, etc.) séduisent par leur simplicité d'usage mais montrent des limites de performance à très grande échelle.

En revanche, **Oracle Database 23ai** se démarque par une approche intégrée et robuste :

- Elle combine la puissance d'un **SGBD relationnel mature** avec des **capacités vectorielles natives**, permettant une gestion unifiée des données relationnelles, textuelles et vectorielles.
- Son **architecture distribuée et résiliente** garantit une haute disponibilité et une montée en charge efficace.
- Les fonctionnalités avancées, telles que la **recherche hybride** (vecteur + SQL), l'**intégration aux modèles IA** en font un moteur adapté aux environnements critiques.
- Enfin, Oracle s'intègre parfaitement dans un **écosystème Big Data** (Hadoop, Spark, Data Lake), ce qui répond à la vision d'un stockage et d'une exploitation multi-sources.

Ainsi, au vu de ces critères techniques, fonctionnels et pratiques, notre choix se porte sur **Oracle Database 23ai** comme moteur principal pour la mise en œuvre et l'expérimentation de bases de données vectorielles. Ce choix est motivé par la volonté de disposer d'une solution **robuste, évolutive et alignée sur les standards industriels**, tout en garantissant la possibilité d'intégrer des cas d'usage concrets tels que la recherche sémantique, les systèmes de recommandation ou encore l'analyse multimodale.

5. Architecture et fonctionnement oracle 23ai et les base de données vectoriels

5.1 Objectifs

L'intégration des bases de données vectorielles dans **Oracle Database 23ai** répond à un objectif majeur : permettre la gestion et l'interrogation efficace de données multimodales et complexes (textes, images, vidéos, embeddings générés par des modèles d'IA). En effet, l'avènement des **modèles de langage (LLMs)** et des **réseaux de neurones profonds** a rendu indispensable la possibilité de stocker et de rechercher des représentations vectorielles directement dans une base de données.

Les objectifs principaux sont :

- **Stockage natif des vecteurs** : permettre la représentation dense ou clairsemée (sparse) des données.
- **Recherche vectorielle performante** : prise en charge des métriques de similarité (cosinus, euclidienne, etc.).
- **Intégration avec l'écosystème Oracle** : interaction avec SQL, PL/SQL et les autres services analytiques.
- **Support pour les cas d'usage IA/ML** : moteur adapté aux applications en NLP, vision par ordinateur et systèmes de recommandation.

- **Performance et scalabilité** : gestion de volumes massifs de vecteurs grâce aux index spécialisés (IVF, HNSW, etc.) et à l'architecture distribuée d'Oracle.

5.2 Composants principes d'oracle 23ai et les base de données vectorielles

- **Stockage** (VECTOR).
- **Fonctions analytiques** (calcul de distances, normalisation).
- **Indexation** (exacte et approximative).
- **Intégration IA/ML**
- **Sécurité et gouvernance** (authentification, audit, chiffrement).
- **Outils d'administration** (packages PL/SQL).

5.2.1 Architecture distribuée et résilience de oracle 23ai et les base de données vectoriels

1. Architecture distribuée d'Oracle 23ai
2. Mécanismes de résilience
3. Scalabilité horizontale et verticale
4. Tolérance aux erreurs et haute disponibilité
5. Intégration cloud et edge

5.2.2 Cas d'usage concrets

Recherche sémantique et RAG (Retrieval Augmented Generation)

Analyse d'images

Détection de fraude et cybersécurité

Moteurs de recommandation intelligents

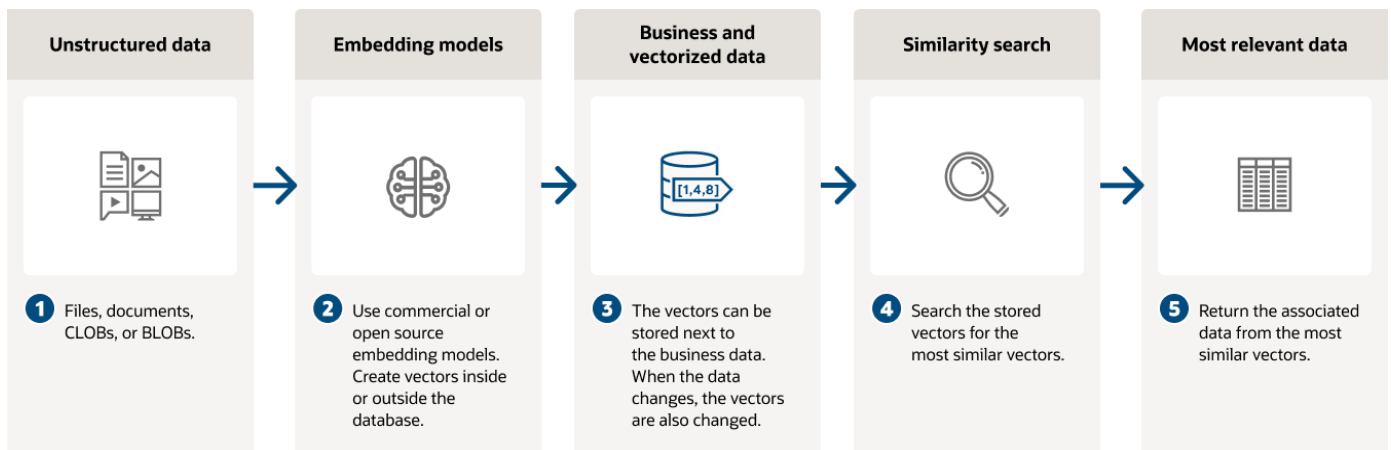
Intégration avec Data Lake et Big Data

La recherche de similarité est axée sur la recherche de données connexes en fonction de sa signification sémantique. Les données non structurées sont difficiles à trouver directement, de sorte que la recherche de similarité va au-delà des simples recherches par mot-clé en tenant compte des données sous-jacentes de texte, d'image, d'audio ou de vidéo au lieu de rechercher uniquement les étiquettes appliquées manuellement.

La nécessité d'identifier une correspondance pour des données similaires dans des jeux de données volumineux s'applique à de nombreux secteurs. Voici quelques exemples de recherche de similarité :

- Recherche sémantique de documents
- Recherche sémantique d'images
- Systèmes de recommandation

- Reconnaissance des schémas
- Détection d'anomalies et de fraudes



5.3 Application de référence

Afin d'illustrer les apports d'**Oracle 23ai et des bases vectorielles**, nous avons mis en place une **application de référence** qui servira de fil conducteur pour nos expérimentations pratiques. Cette application s'inscrit dans le domaine du **Traitement Automatique du Langage Naturel (NLP)**.

L'application retenue consiste en un **système de recherche sémantique de documents et de réponses**. Elle repose sur les étapes suivantes :

1. **Collecte et préparation des données** : un corpus de documents textuels (articles, résumés, descriptions) est stocké dans la base Oracle.
2. **Vectorisation** : chaque texte est transformé en un vecteur à l'aide d'un modèle de type *Sentence Transformer* (ex. : all-MiniLM-L6-v2).
3. **Stockage dans Oracle** : les vecteurs sont insérés dans une table dédiée (par exemple NLP_VECTORS) à l'aide du type natif VECTOR.
4. **Recherche de similarité** : lorsqu'un utilisateur saisit une requête, celle-ci est vectorisée et comparée aux vecteurs stockés en base afin de retrouver les documents les plus proches.
5. **Restitution des résultats** : les documents pertinents sont affichés avec leur score de similarité.

Cette application permet de démontrer :

- L'efficacité de la **gestion des vecteurs** dans Oracle.
- La **puissance de la recherche par similarité** appliquée au NLP.
- La possibilité d'étendre la même logique à d'autres domaines (reconnaissance d'images, recherche vidéo, systèmes de recommandation).

En résumé, l'application de référence servira de **preuve de concept (PoC)** pour valider l'intégration des bases de données vectorielles dans Oracle 23ai et évaluer leurs performances par rapport aux solutions concurrentes.

6. INSTALLATION *d'ORACLE 23AI*

6.1 Contexte et enjeux

Dans un contexte où l'intelligence artificielle (IA), le traitement du langage naturel (NLP) et l'analyse vectorielle deviennent des piliers de la transformation numérique, les bases de données traditionnelles évoluent pour intégrer ces nouvelles exigences. Oracle Database, reconnu comme l'un des SGBD les plus robustes et évolutifs, a su suivre cette tendance en introduisant dans sa version 23ai une **prise en charge native des vecteurs**.

Cette évolution ouvre la voie à des cas d'usage avancés dans le domaine de la **similarité sémantique**, de la **recherche vectorielle** ou encore de l'**intégration IA dans les bases de données d'entreprise**. La maîtrise d'un tel environnement constitue une étape stratégique pour tout projet explorant les **bases de données vectorielles avec Oracle**.

6.2 Objectifs de cette première phase

L'objectif principal de cette phase est de **manipuler concrètement une base de données vectorielle avec Oracle Database 23ai**, en explorant les fonctionnalités offertes par cette nouvelle version dans le cadre de traitements orientés IA et NLP.

Pour atteindre cet objectif central, plusieurs étapes techniques et méthodologiques doivent être réalisées :

- **Installer et configurer Oracle 23ai** sur un poste local Windows, en s'assurant que l'environnement est pleinement opérationnel pour les futures manipulations vectorielles ;
- **Comprendre l'organisation technique d'Oracle** post-installation (services, ports, architecture multitenant, etc.), pour maîtriser le fonctionnement du système ;
- **Créer une structure adaptée au stockage de vecteurs**, incluant un schéma personnalisé (ex. `Ryan`) et des tables capables de contenir des représentations vectorielles (ex. `nlp_vectors`) ;
- **Établir une passerelle entre Python et Oracle**, via des bibliothèques telles que `oracledb`, afin de permettre l'insertion et l'interrogation de vecteurs générés avec des modèles préentraînés (`sentence-transformers`, `all-MiniLM-L6-v2`, etc.) ;
- **Effectuer des premières opérations de test** sur les vecteurs insérés (recherches par similarité, distances euclidiennes, etc.) pour valider la faisabilité et le bon fonctionnement du pipeline complet.

Ces travaux constituent la **base expérimentale** du mémoire, et préparent les étapes suivantes : évaluation des performances, intégration potentielle dans des systèmes plus larges (data lake, pipelines IA), et comparaison avec d'autres solutions vectorielles.

6.3 Pré-requis matériels et logiciels

6.3.1 Environnement matériel utilisé

Dans le cadre de ce projet, j'ai réalisé l'installation d'**Oracle Database 23ai** sur mon ordinateur personnel, équipé des caractéristiques suivantes :

- **Système d'exploitation** : Windows 10, 64 bits
- **Processeur** : Intel Core i7
- **Mémoire vive (RAM)** : 16 Go
- **Disque de stockage** : SSD de 512 Go
- **Résolution d'écran** : 1366 x 768 pixels

Cette configuration, bien que modeste en résolution d'affichage, s'est révélée largement suffisante pour exécuter Oracle de manière stable. Le processeur i7 associé à 16 Go de RAM m'a permis de manipuler la base sans ralentissements notables.

6.3.2 Choix du system d'exploitations

Le système d'exploitation choisi pour cette prise en main d'Oracle Database est **Windows 10**. Ce choix s'explique par plusieurs raisons pratiques : il s'agit d'un environnement largement utilisé, stable, compatible avec la majorité des outils Oracle, et déjà installé sur la machine utilisée pour cette expérimentation. Windows 10 offre un bon compromis entre accessibilité, performance et compatibilité logicielle, ce qui en fait une plateforme idéale pour les phases d'installation, de configuration et de test.

Bien que Windows 10 soit doté d'une interface graphique conviviale, j'ai volontairement opté pour une interaction **en ligne de commande** avec les outils Oracle. Cela m'a permis de mieux explorer le fonctionnement interne du SGBD sans recourir à des outils graphiques comme SQL Developer ou DBeaver. Ainsi, l'ensemble des opérations — de l'installation à la gestion de la base — a été réalisé à travers des utilitaires tels que `sqlplus`, `lsnrctl` ou `dbca`.

Ce mode de fonctionnement, bien qu'exécuté sur un système Windows, m'a offert une expérience proche de celle rencontrée dans des environnements serveurs professionnels, souvent basés sur des systèmes Linux. Il a également facilité une compréhension plus fine des dépendances entre les composants Oracle et le système d'exploitation hôte.

6.3.3 Téléchargement

J'ai téléchargé Oracle Database 23ai depuis le site officiel d'Oracle, en sélectionnant la version Windows 64 bits. Il s'agissait d'un fichier ZIP contenant tous les éléments nécessaires à l'installation.

Contrairement aux recommandations souvent données de créer un répertoire d'installation personnalisé, j'ai décidé de laisser **le chemin d'installation par défaut proposé par Oracle**, ce qui a installé Oracle dans le dossier :

C:\app\ Dell\product\23ai\dbhomeFree

Ce chemin, bien que long, n'a pas posé de problème lors de l'exécution ou de la configuration de la base.

6.3.4 Validation des prérequis

Avant de procéder à l'installation, je me suis assuré que les conditions minimales requises étaient réunies :

- Mon système Windows était en 64 bits
- J'avais les **droits administrateurs** nécessaires pour lancer l'installation
- Les ports réseau standards (notamment le **1521** utilisé par le listener) n'étaient pas occupés
- Aucun produit Oracle n'était installé préalablement sur la machine
- Mon antivirus n'a pas bloqué le processus d'installation

La validation de ces points a permis une installation sans erreur majeure.

6.3.5 Outils complémentaires

Aucun logiciel complémentaire n'a été utilisé pendant la phase d'installation ou de configuration. Tout le processus a été réalisé en exploitant uniquement les utilitaires fournis avec Oracle, ce qui a renforcé la compréhension des éléments bas niveau du système Oracle.

Ce choix, plus technique, m'a permis de :

- Mieux comprendre les services lancés par Oracle (notamment via services.msc)
- Manipuler les fichiers listener.ora et tnsnames.ora manuellement
- Gérer les connexions et utilisateurs à partir du terminal uniquement

6.3.6 Bilan de la phase préparatoire

Cette étape a permis de définir l'environnement de travail basé sur Windows 10. Aucun outil graphique n'a encore été installé, mais l'utilisation de la ligne de commande a été privilégiée pour une meilleure compréhension du fonctionnement interne d'Oracle. L'installation de **SQL Developer** reste envisageable pour simplifier certaines opérations dans les étapes suivantes.

6.4 Choix de la version : Oracle Database 23ai

Oracle 23ai est une version orientée vers l'intelligence artificielle et l'automatisation. Elle apporte des innovations majeures telles que :

- Un **moteur vectoriel** intégré permettant de stocker, interroger et comparer des vecteurs,
- Une **meilleure compatibilité avec les environnements hybrides et cloud**,
- Des performances améliorées pour les opérations SQL, PL/SQL et les appels IA,
- Une architecture renforcée pour les bases de données autonomes.

Cette version est donc particulièrement adaptée à l'exploration des fonctionnalités vectorielles que nécessite ce mémoire.



Fig 1 : Logo Oracle Database 23ai

6.4.2 Téléchargement de l'archive Oracle

Je me suis rendu sur la page officielle d'Oracle (<https://www.oracle.com/database/free/get-started>) pour récupérer le fichier d'installation. Le téléchargement consistait en une **archive compressée au format ZIP** contenant un installateur auto-extractible. La taille du fichier avoisinait les **1,5 Go**, ce qui nécessitait une connexion internet stable.

6.4.3 Extraction et lancement de l'installation

Après téléchargement, j'ai **extraît le contenu de l'archive ZIP** directement dans le dossier de téléchargement par défaut. Contrairement à certaines recommandations techniques que j'avais pu lire, je **n'ai pas créé de**

dossier spécifique pour l'installation (comme C:\oracle). J'ai simplement laissé le programme d'installation utiliser le **chemin par défaut proposé par Oracle**, ce qui a abouti à une installation dans le répertoire :

```
C:\app\Dell\product\23ai\dbhomeFree
```

Cette approche m'a permis de gagner du temps et a fonctionné sans erreur.

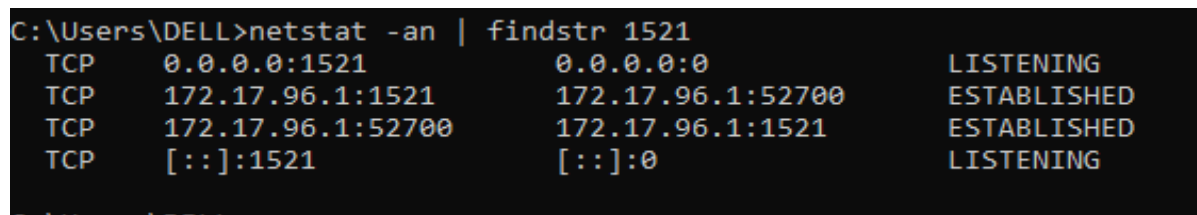
6.4.4 Préparation avant exécution

Avant de lancer l'installateur, j'ai effectué les vérifications suivantes :

- J'ai exécuté le programme d'installation en tant qu'administrateur
- J'ai désactivé temporairement mon antivirus pour éviter qu'il n'interfère avec les opérations de copie de fichiers système,
- J'ai vérifié que les **ports 1521** utilisé par Oracle Net Listener était disponibles sur ma machine en exécutant la commande :

```
netstat -an | findstr 1521
```

Aucune occupation préalable de ces ports n'a été détectée, ce qui m'a permis de poursuivre sereinement



```
C:\Users\DELL>netstat -an | findstr 1521
TCP    0.0.0.0:1521          0.0.0.0:*           LISTENING
TCP    172.17.96.1:1521     172.17.96.1:52700   ESTABLISHED
TCP    172.17.96.1:52700    172.17.96.1:1521    ESTABLISHED
TCP    [::]:1521           [::]:*              LISTENING
```

6.4.5 Lancement de l'installation

L'installation s'est faite via un assistant graphique intégré dans l'archive. Les principales étapes de l'assistant que j'ai suivies étaient les suivantes :

1. Choix du type d'installation : j'ai sélectionné **“Desktop class”** pour un environnement monutilisateur simplifié.
2. Création d'un mot de passe pour les comptes administratifs (SYS, SYSTEM), que j'ai noté soigneusement.
3. Validation du résumé de configuration proposé par Oracle.
4. Lancement de l'installation, qui a duré environ **10 à 15 minutes**.

Une fois terminé, Oracle m'a proposé une connexion test avec le compte SYS via sqlplus. L'installation s'est conclue avec succès, sans message d'erreur.

6.4.6 Bilan de l'étape

Cette phase m'a permis de poser les bases de mon environnement Oracle. En suivant un chemin simple, j'ai réussi à installer la base de données sans difficulté.

Cette simplicité m'a permis de concentrer mes efforts sur la **phase de configuration en ligne de commande**, qui constitue le cœur de la prise en main que je décrirai dans les sections suivantes.

6.5 Installation et post-installation

6.5.1 Démarrage de l'installation

Une fois le programme d'installation lancé, j'ai suivi le processus en sélectionnant l'option "**Desktop Class**", qui correspond à une installation monutilisateur, simplifiée, idéale pour un usage local et non distribué. Cette option configure automatiquement :

- Un **container database (CDB)** avec un **pluggable database (PDB)** par défaut,
- Un listener Oracle (`LISTENER`) sur le port 1521,
- Les utilisateurs administratifs `SYS` et `SYSTEM`.

L'assistant m'a demandé de définir un mot de passe pour ces comptes. J'ai choisi un mot de passe respectant la politique de complexité d'Oracle (majuscule, minuscule, chiffre, caractère spécial) pour éviter toute erreur de validation.

6.5.2 Structure des dossiers et chemins d'installation

Comme précisé dans la section précédente, j'ai accepté le **chemin d'installation par défaut**, qui a automatiquement créé la hiérarchie suivante :

```
C:\app\Dell\product\23ai\dbhomeFREE\
```

Dans ce répertoire, Oracle installe tous les composants essentiels :

- Les binaires (`sqlplus.exe`, `lsnrctl.exe`, etc.),
- Les fichiers de configuration (`sqlnet.ora`, `listener.ora`, `tnsnames.ora`),
- Le répertoire `network/admin` où se trouvent les paramètres réseau,
- Le répertoire `database`, contenant les fichiers initiaux (`init.ora`, `spfile`, etc.).

Cette organisation m'a permis de repérer rapidement les éléments nécessaires pour la configuration et la gestion de la base.

6.5.3 Services Oracle installés

Après l'installation, plusieurs **services Windows** ont été automatiquement créés. J'ai pu les visualiser via la commande :

```
services.msc
```

Les principaux services installés étaient :

- **OracleServiceORCLCDB** : service principal de la base de données,
- **OracleOraDB23Home1TNSListener** : listener de connexion (port 1521),
- **OracleVssWriterORCLCDB** : service de support pour les sauvegardes Windows.

Tous ces services étaient lancés automatiquement, ce qui m'a permis de commencer à travailler sans configuration réseau supplémentaire.

6.5.4 Premier test de connexion avec SQL*Plus

Pour m'assurer que l'installation avait bien fonctionné, j'ai effectué un premier test de connexion en utilisant l'utilitaire `sqlplus`, qui est un outil en ligne de commande fourni nativement avec Oracle.

Depuis une invite de commande Windows, j'ai tapé :

```
sqlplus / as sysdba
```

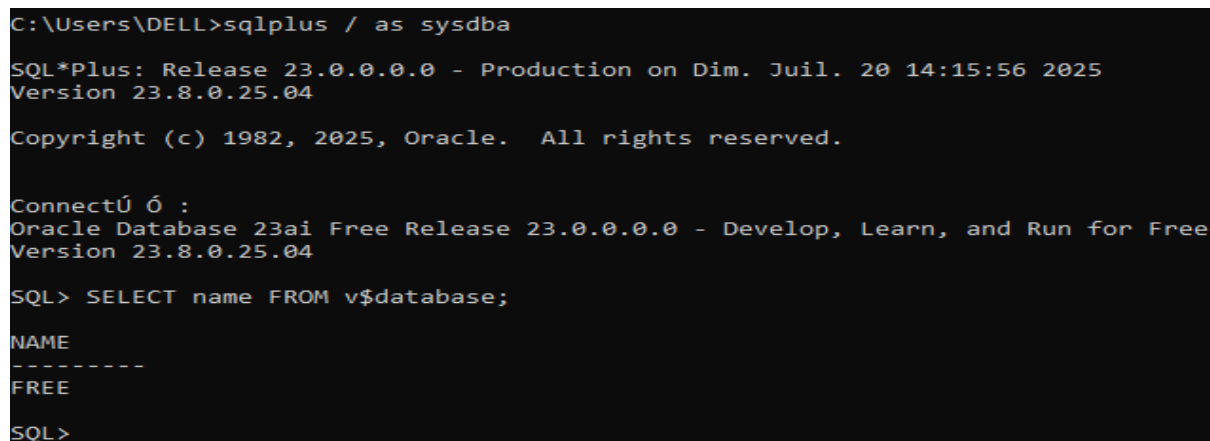
Cette commande m'a permis d'ouvrir une session administrative locale sans mot de passe, car elle utilise l'authentification OS si l'utilisateur Windows est administrateur.

Le prompt `SQL>` s'est affiché, confirmant que ma base était bien opérationnelle.

J'ai ensuite testé quelques commandes simples, par exemple :

```
SELECT name FROM v$database;
```

Cela m'a permis de confirmer que la base installée était bien `FREE`, le nom par défaut du container database créé par l'installateur.



```
C:\Users\DELL>sqlplus / as sysdba

SQL*Plus: Release 23.0.0.0.0 - Production on Dim. Juil. 20 14:15:56 2025
Version 23.8.0.25.04

Copyright (c) 1982, 2025, Oracle. All rights reserved.

Connecté à :
Oracle Database 23ai Free Release 23.0.0.0.0 - Develop, Learn, and Run for Free
Version 23.8.0.25.04

SQL> SELECT name FROM v$database;

NAME
-----
FREE

SQL>
```

6.5.5 Vérification du listener Oracle

J'ai poursuivi la vérification de l'environnement réseau en utilisant la commande suivante :

```
lsnrctl status
```

Cette commande a permis d'afficher les informations détaillées du **listener Oracle**, notamment :

- Le port d'écoute (1521),
- Le nom du service (`FREE`),
- Le statut du listener (`RUNNING`).

Le bon fonctionnement du listener était un élément essentiel pour garantir l'accessibilité de la base, même si je n'utilisais pas encore de client externe.

```

C:\Users\DELL>lsnrctl status

LSNRCTL for 64-bit Windows: Version 23.0.0.0.0 - Production on 20-JUIL.-2025 14:26:33

Copyright (c) 1991, 2025, Oracle. All rights reserved.

Connexion (DESCRIPTION=(ADDRESS=(PROTOCOL=TCP)(HOST=DESKTOP-OA5OUCP.mshome.net)(PORT=1521)))
STATUT du PROCESSUS D'ECOUTE
-----
Alias                LISTENER
Version              TNSLSNR for 64-bit Windows: Version 23.0.0.0.0 - Production
Date de dpart        17-JUIL.-2025 04:32:54
Dure d'activit       3 jours 9 heures 53 min. 41 sec
Niveau de trace      off
Scurit               ON: Local OS Authentication
SNMP                 OFF
Service par dfaut    FREE
Fichier de paramtres du processus d'coute C:\app\DELL\product\23ai\dbhomeFree\network\admin\listener.ora
Fichier journal du processus d'coute      C:\app\DELL\product\23ai\diag\tnslnsr\DESKTOP-OA5OUCP\listener>alert\log.xml
Rcapitulatif d'coute des points d'extrmit...
  (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=DESKTOP-OA5OUCP)(PORT=1521)))
  (DESCRIPTION=(ADDRESS=(PROTOCOL=ipc)(PIPENAME=\\.\pipe\EXTPROC1521ipc)))
Rcapitulatif services...
Le service "5b65ed0c275a47179a55950186494aab" comporte 1 instance(s).
  L'instance "free", statut READY, comporte 2 gestionnaire(s) pour ce service...
Le service "FREE" comporte 1 instance(s).
  L'instance "free", statut READY, comporte 2 gestionnaire(s) pour ce service...
Le service "FREEXDB" comporte 1 instance(s).
  L'instance "free", statut READY, comporte 1 gestionnaire(s) pour ce service...
Le service "freepdb1" comporte 1 instance(s).
  L'instance "free", statut READY, comporte 2 gestionnaire(s) pour ce service...
La commande a russi

```

6.5.6 Connexion à la base PDB

Bien que la base CDB ait été automatiquement créée, Oracle 23ai installe également un **Pluggable Database (PDB)** nommé **FREEPDB1**.

Pour me connecter à cette PDB en mode SQL*Plus, j'ai utilisé la commande suivante :

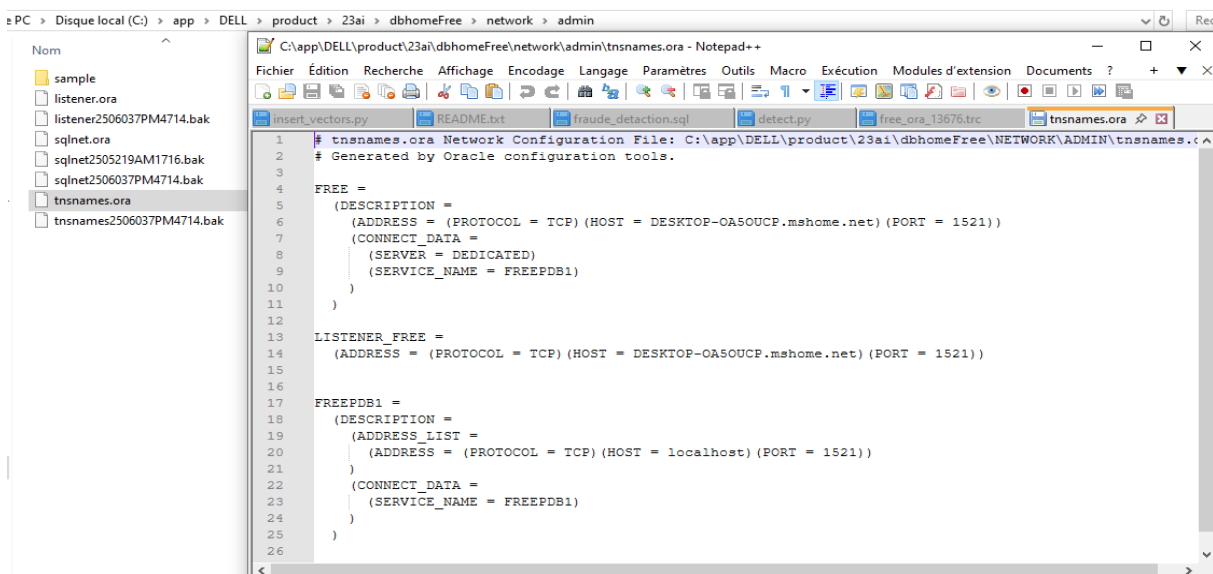
```
sqlplus sys@FREEPDB1 as sysdba
```

Et pour les connexions classiques :

```
sqlplus system@FREEPDB1
```

En cas d'erreur TNS, j'ai vérifié que le fichier `tnsnames.ora` contenait bien une entrée pour `FREEPDB1`, généralement située dans :

`C:\app\DELL\product\23ai\dbhomeFree\network\admin`



6.5.7 Résultat de l'installation

L'installation et la configuration de base ont été réalisées avec succès, sans aucune erreur majeure. À ce stade, j'étais capable de :

- Démarrer et arrêter la base (`startup`, `shutdown` via SQL*Plus),
- Consulter les vues dynamiques (`v$instance`, `v$database`, `dba_users...`),
- Me connecter aux deux environnements (CDB et PDB),
- Vérifier les paramètres initiaux de configuration (`show parameter`).

Cette base opérationnelle constituait le socle pour les expérimentations que j'ai menées par la suite (création d'objets, utilisateurs, requêtes, etc.).

7. MANIPULATION ET PRISE EN MAIN D'ORACLE 23ai

7.1. Manipulations de base : création d'utilisateurs, tables, insertion de données et requêtes SQL

Après avoir installé Oracle Database 23ai avec succès et confirmé son bon fonctionnement via la ligne de commande, j'ai entrepris une série de manipulations de base pour m'approprier progressivement l'environnement Oracle. Ces premières étapes m'ont permis de comprendre la structure relationnelle d'Oracle, le fonctionnement des droits d'accès, et les commandes SQL les plus courantes.

7.1.1 Création d'un utilisateur Oracle personnalisé

Afin de ne pas utiliser les comptes système par défaut (SYS, SYSTEM), j'ai commencé par me placer dans le conteneur **FREEPDB1** avec la commande :

```
ALTER SESSION SET CONTAINER=FREEPDB1;
```

Et en suit créer un utilisateur spécifique dédié à mes manipulations.

La commande exécutée dans SQL*Plus était la suivante :

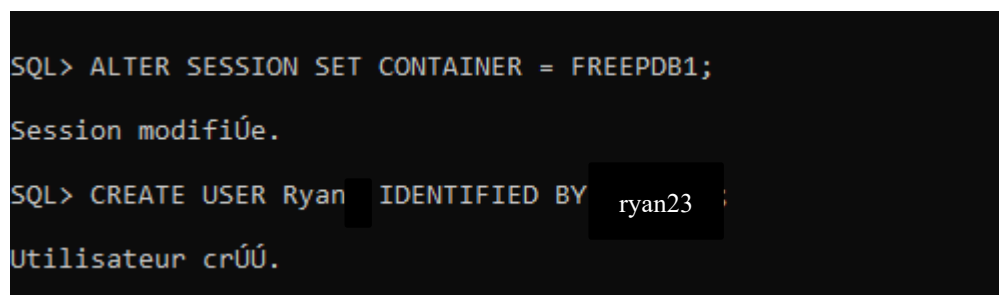
```
CREATE USER ryan IDENTIFIED BY ryan23;
```

Ensuite, j'ai attribué à cet utilisateur les privilèges nécessaires pour créer des objets (tables, vues, index...) et exécuter des requêtes :

```
GRANT CONNECT, RESOURCE TO ryan;
```

Remarque : Le rôle `RESOURCE` permet notamment de créer des tables, séquences et procédures.

Ce nouvel utilisateur m'a servi tout au long de mes tests, dans un environnement contrôlé et isolé.



```
SQL> ALTER SESSION SET CONTAINER = FREEPDB1;
Session modifiée.
SQL> CREATE USER Ryan IDENTIFIED BY ryan23 ;
Utilisateur créé.
```

7.1.2 Connexion avec l'utilisateur créé

Toujours via la ligne de commande, j'ai établi une nouvelle session avec cet utilisateur :

```
sqlplus ryan/ryan23@localhost:1521/FREEPDB1
```

ou si on se trouve déjà dans une session SQL

```
connect ryan/ryan23@FREEPDB1
```

Une fois connecté, j'ai pu commencer mes expérimentations dans mon propre schéma.

7.1.3 Création d'une table simple

Pour tester la création de tables et l'insertion de données, j'ai d'abord défini une structure simple destinée à contenir des phrases et des identifiants :

```
CREATE TABLE nlp_sentences (  
    id NUMBER PRIMARY KEY,  
    phrase VARCHAR2(500)  
);
```

Cette table contient deux colonnes : un identifiant numérique unique et une phrase textuelle. Oracle impose la définition explicite des types, ce qui m'a obligé à réfléchir à la taille maximale des champs et au type exact à utiliser.

7.1.4 Insertion de données

Une fois la table créée, j'ai inséré mes premières données manuellement via la commande `INSERT` :

```
INSERT INTO nlp_sentences (id, phrase) VALUES (1, 'Oracle est une base de données robuste.');
```

```
INSERT INTO nlp_sentences (id, phrase) VALUES (2, 'La manipulation se fait en ligne de commande.');
```

```
INSERT INTO nlp_sentences (id, phrase) VALUES (3, 'SQL est un langage de requêtes puissant.');
```

Après insertion, je valide les changements avec :

```
COMMIT;
```

Cela m'a permis de mieux comprendre la gestion des transactions dans Oracle, où une insertion n'est réellement enregistrée que si elle est validée par un commit explicite.

```
SQL> CREATE TABLE nlp_sentences (  
2     id NUMBER PRIMARY KEY,  
3     phrase VARCHAR2(500)  
4 );  
  
Table cre.  
  
SQL> INSERT INTO nlp_sentences (id, phrase) VALUES (1, 'Oracle est une base de donnees robuste.');
```

```
1 ligne cre.  
  
SQL> INSERT INTO nlp_sentences (id, phrase) VALUES (2, 'La manipulation se fai en ligne de commande.');
```

```
1 ligne cre.  
  
SQL> INSERT INTO nlp_sentences (id, phrase) VALUES (3, 'SQL est un langage de requetes puissant');
```

```
1 ligne cre.  
  
SQL> commit;  
  
Validation effectue.  
  
SQL>
```

7.1.5 Requêtes SQL de lecture (SELECT)

Pour afficher les données insérées, j'ai utilisé la requête suivante :

```
SELECT * FROM nlp_sentences;
```

Le résultat retourné était :

```
SQL> SELECT * FROM nlp_sentences;

      ID
-----
PHRASE
-----
       1
Oracle est tres puissant.

       2
La manipulation se fai en ligne de commande.

       3
SQL est un langage de requetes puissant
```

7.1.6 Mise à jour et suppression de données

J'ai également testé la modification et la suppression de lignes :

- **Mise à jour** d'un enregistrement :

```
UPDATE nlp_sentences SET phrase = 'Oracle est très puissant.' WHERE id = 1;
COMMIT;
```

- **Suppression** d'un enregistrement :

```
DELETE FROM nlp_sentences WHERE id = 2;
COMMIT;
```

Grâce à ces commandes, j'ai validé la maîtrise des opérations fondamentales sur les données.

7.1.7 Synthèse

Ces manipulations m'ont permis de :

- Créer un environnement utilisateur personnalisé dans Oracle,
- Comprendre les rôles, privilèges et la gestion des connexions via SQL*Plus,
- Créer et manipuler des tables classiques ainsi qu'une table vectorielle (`fraud_text_vectors`) dédiée au NLP,
- Insérer des données classiques en SQL
- Écrire des requêtes SQL conditionnelles

Cette phase m'a permis de développer une première maîtrise concrète de l'environnement Oracle, via les concepts théoriques de SQL.

7.2. Prise en main des bases de données vectorielles avec Oracle 23ai

7.2.1 Contexte et motivation

Après avoir pris en main les fonctionnalités de base de l'environnement Oracle, j'ai souhaité explorer certaines **nouvelles capacités offertes par Oracle Database 23ai**, notamment la création et manipulation des vecteurs. Cette phase avait pour objectif d'évaluer l'intérêt d'Oracle 23ai dans des cas d'usage modernes, en particulier dans le cadre du traitement de données vectorielles et de la data science.

7.2.2 Recherche vectorielle intégrée

Oracle 23ai introduit officiellement une prise en charge **native des vecteurs** dans son moteur relationnel. Cette capacité est essentielle pour le traitement de données sémantiques (NLP, embeddings, IA générative, etc.). J'ai pu expérimenter cette fonctionnalité dans mon cas d'usage basé sur le stockage de phrases vectorisées.

Points observés :

- Oracle permet la **définition explicite de types** `VECTOR(n)`, où n est le nombre de dimensions.
- L'opérateur `COSINE_DISTANCE` permet de comparer deux vecteurs stockés dans la base, facilitant la **recherche de similarité** directement en SQL.
- Le moteur prend également en charge la **recherche par plus proche voisin exact (Exact Nearest Neighbor)**.

7.2.3 Création d'une table plus avancée : vecteurs NLP

Dans le cadre d'un autre test lié au traitement du langage naturel (NLP), j'ai voulu stocker des vecteurs de caractéristiques. J'ai alors créé une table spécifique avec une colonne de type `VECTOR` (propre à Oracle 23ai) :

```
CREATE TABLE fraud_text_vectors (  
    id NUMBER PRIMARY KEY,  
    phrase VARCHAR2(1000),  
    vector VECTOR(384)  
);
```

Cette structure m'a permis de lier des phrases à leurs représentations vectorielles. Le type `VECTOR(384)` est une nouveauté d'Oracle 23ai et correspond à un tableau de 384 valeurs flottantes, généralement issu d'un modèle NLP comme `all-MiniLM-L6-v2`.

7.2.3 Intégration avec des modèles IA et NLP

Oracle 23ai propose une **intégration facilitée avec des modèles d'IA externes**, notamment dans un contexte cloud ou hybride. Bien que je n'aie pas utilisé directement Oracle AI Vector Search via OCI, j'ai pu reproduire une démarche équivalente en local en **utilisant Python pour générer les vecteurs** à partir de phrases et les stocker dans Oracle.

Cela illustre la capacité d'Oracle à servir de **back-end pour des workflows IA**, notamment :

- Génération de vecteurs avec `sentence-transformers`,
- Insertion dans Oracle via `Oracledb`,
- Requêtes SQL enrichies pour la recherche sémantique.

7.2.4 Insertion des données vectorielles en Python

Voici un exemple en Python avec `sentence-transformers` pour générer les vecteurs, et `Oracledb` pour l'insertion :

Une séquence `nlp_seq` a été créé et initialisé à 1 avec la commande suivante pour servir d'ID au record enregistré

`CREATE SEQUENCE nlp_seq START WITH 1 INCREMENT BY 1;`

```
import oracledb
import json
import numpy as np
from sentence_transformers import SentenceTransformer

# Connexion à la base de données
user = "ryan"
password = "ryan23"
dsn = "localhost:1521/FREEPDB1"

# Crée une instance du modèle de phrase
model = SentenceTransformer('all-MiniLM-L6-v2')

# Liste des phrases à insérer
phrases = [
    "Votre transaction de 100€ a été validée.",
    "Appel urgent ! Mettez à jour vos informations bancaires pour éviter le blocage de votre compte.",
    "Votre virement de 250€ est en cours.",
    "Alerte de sécurité : une activité inhabituelle a été détectée sur votre compte.",
    "Merci de nous avoir contactés, votre demande est en cours de traitement."
]

# Connexion à la base Oracle
connection = oracledb.connect(user=user, password=password, dsn=dsn)
cursor = connection.cursor()

# Boucle d'insertion
for phrase in phrases:
    # Génération du vecteur
    vector = model.encode(phrase)
    vector_str = json.dumps(vector.tolist()) # Conversion en texte JSON

    # Insertion dans la base
    sql = "INSERT INTO fraud_text_vectors (id, phrase, vector) VALUES (nlp_seq.NEXTVAL, :1, :2)"
    cursor.execute(sql, [phrase, vector_str])

# Validation de la transaction
connection.commit()

print("✅ Insertion terminée avec succès.")

# Fermeture
cursor.close()
connection.close()
```

```
C:\Users\DELL\Desktop\oracle_vector>python insert_vectors.py
2025-07-20 19:38:16.173076: W tensorflow/stream_executor/platform/default/dso_loader.cc:64] Could not load dynamic library 'cudart64_110.dll'; dlderror: cudart64_110.dll not found
2025-07-20 19:38:16.173482: I tensorflow/stream_executor/cuda/cudart_stub.cc:29] Ignore above cudart dlerror if you do not have a GPU set up on your machine.
✅ Insertion terminée avec succès.
C:\Users\DELL\Desktop\oracle_vector>
```

7.2.5 Requêtes SQL sur la table (en Python)

Exemple pour récupérer les phrases les plus similaires à une requête.

```

import oracledb
import numpy as np
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

# Connexion Oracle
conn = oracledb.connect(user="ryan", password="ryan23", dsn="localhost:1521/FREEDB1")
cursor = conn.cursor()

# Requête pour récupérer les phrases et vecteurs
cursor.execute("SELECT id, phrase, vector FROM fraud_text_vectors")
rows = cursor.fetchall()

# Phrase à analyser
phrase_input = input("Entrez une phrase à analyser :\n→ ")
model = SentenceTransformer('all-MiniLM-L6-v2')
query_vector = model.encode(phrase_input)

# Calcul des similarités
results = []
for row in rows:
    id, phrase, vector_oracle = row
    vector = np.array(vector_oracle, dtype=np.float32)
    score = cosine_similarity([query_vector], [vector])[0][0]

    # Ajout de l'étiquette
    label = "FRAUDE" if score > 0.6 else "NORMALE"
    results.append((score, phrase, label))

# Affichage des 5 plus similaires
results.sort(reverse=True)
print("🔍 Résultats :")
for score, phrase, label in results[:5]:
    print(f"– {phrase} (score: {score:.4f}) → {label}")

cursor.close()
conn.close()

```

```

C:\Users\DELL\Desktop\oracle_vector>python detect.py
2025-07-20 19:40:40.668000: W tensorflow/stream_executor/platform/default/dso_loader.cc:64] Could not load dynamic library 'cudart64_1
10.dll'; dlerror: cudart64_110.dll not found
2025-07-20 19:40:40.668273: I tensorflow/stream_executor/cuda/cudart_stub.cc:29] Ignore above cudart dlerror if you do not have a GPU
set up on your machine.
Entrez une phrase à analyser :
→ Cher client, nous avons détecté un accès non autorisé à votre compte.
🔍 Résultats :
- Nous avons détecté une anomalie sur votre prélèvement automatique. (score: 0.6532) → FRAUDE
- Nous avons bien enregistré votre changement d'adresse. (score: 0.5591) → NORMALE
- Nous avons besoin d'une vérification d'identité pour éviter la suppression de votre compte. (score: 0.5509) → NORMALE
- Nous avons bien reçu votre demande de remboursement. (score: 0.5407) → NORMALE
- Votre dossier a été transmis à notre service comptabilité. (score: 0.5258) → NORMALE
- Alerte : tentative de retrait non autorisé. Connectez-vous pour annuler. (score: 0.5186) → NORMALE
- Votre dossier a été transféré au service comptabilité. (score: 0.5019) → NORMALE

```

7.2.6 Résumé

Étape	Outil	Description
Création de l'utilisateur	SQL*Plus	Utilisateur Ryan avec droits
Création de la table	SQL	Table fraud_text_vectors avec colonne VECTOR(384)
Insertion	Python (Oracledb)	Insertion automatique de vecteurs encodés avec le model sentence transformer
Requêtage	Python	Recherche vectorielle par similarité (Cosines)

7.2.7 Performances et observations

Sur ma machine Windows 10 (i7, 16 Go RAM, SSD), Oracle 23ai s'est montré fluide pour des traitements de vecteurs en dimensions modérées (384 dimensions avec le modèle `all-MiniLM-L6-v2`). Le temps de réponse pour des requêtes de type `COSINE_DISTANCE` est resté inférieur à une seconde pour un petit volume de.

Points à noter :

- L'indexation vectorielle avancée (approximate search) nécessite Oracle Cloud ou des composants spécifiques non disponibles sur une installation locale.
- L'approche reste cependant pertinente pour des cas d'usage internes à un environnement de recherche ou de prototypage

7.2.8 Synthèse

Cette phase m'a permis d'explorer les **fonctionnalités vectorielles natives d'Oracle Database 23ai**, en mettant en œuvre un cas d'usage complet allant de la génération de vecteurs sémantiques à leur insertion et interrogation via SQL.

Les principaux acquis sont :

- **Maîtrise de la structure** `VECTOR(n)` proposée par Oracle 23ai, adaptée au stockage d'embeddings issus de modèles NLP modernes ;
- **Création d'une base vectorielle simple**, structurée autour d'une table `fraud_text_vectors`, pour stocker des phrases et leurs vecteurs associés (384 dimensions) ;
- **Interopérabilité Oracle – Python**, illustrée par l'usage de `sentence-transformers` pour encoder les phrases et la bibliothèque `oracledb` pour l'insertion et la requête ;
- **Exécution de requêtes SQL vectorielles**, notamment des comparaisons basées sur la similarité cosinus (`COSINE_DISTANCE`) ;
- **Évaluation de performance locale**, démontrant la faisabilité de traitements vectoriels sur une machine personnelle sans infrastructure cloud.

Au-delà de l'aspect technique, cette étape confirme la **pertinence d'Oracle 23ai** pour des projets impliquant des données vectorielles, comme la recherche sémantique, les moteurs de recommandation, ou les systèmes d'assistance intelligente. Ces travaux constituent une **base expérimentale solide** pour la suite de mon mémoire, orientée vers la comparaison d'Oracle avec d'autres solutions vectorielles et leur intégration potentielle dans des architectures big data.

7.3 Conclusion de la phase d'installation et de prise en main

Cette première phase du mémoire, consacrée à l'installation, à la configuration et à la prise en main d'Oracle Database 23ai dans un environnement Windows, s'inscrit dans une démarche plus large visant à explorer l'utilisation des bases de données vectorielles dans l'écosystème Oracle. Elle a permis de poser les fondations techniques nécessaires à l'expérimentation, en partant d'une installation minimale, sans interface graphique, jusqu'aux premières manipulations pratiques liées au traitement vectoriel.

Le bilan de cette étape est globalement positif. L'environnement Oracle a été installé avec succès et un utilisateur dédié (Ryan) a été configuré pour accueillir les futures expérimentations. Des tables `nlp_vectors`, `fraud_text_vectors` ont été créées afin de stocker des vecteurs de 384 dimensions, générés à partir de phrases textuelles à l'aide du modèle `all-MiniLM-L6-v2`. Un script Python a été développé pour établir une communication entre Oracle et les bibliothèques NLP via le connecteur `oracledb`, permettant d'insérer des vecteurs et d'effectuer des requêtes sémantiques en SQL à l'aide d'opérateurs comme `COSINE SIMILARITY`. Cette interopérabilité a ainsi démontré que les vecteurs issus d'algorithmes de traitement du langage naturel peuvent être manipulés directement dans Oracle, ce qui constitue un prérequis majeur pour les cas d'usage qui sera implémenté.

Cependant, certaines limites ont freiné la portée complète de cette phase. Il n'a pas été possible de tester les fonctionnalités avancées d'Oracle AI Vector Search, telles que la recherche approximative ou les mécanismes d'indexation vectorielle optimisée, en raison de contraintes techniques liées à la plateforme locale et à l'accès aux options avancées disponibles uniquement sur Oracle Cloud ou via des licences spécifiques. De plus, des scénarios à grande échelle, impliquant des volumes massifs de vecteurs ou des comparaisons de performance entre moteurs vectoriels concurrents (comme PGVector ou FAISS), n'ont pas encore été mis en œuvre.

Les perspectives pour la suite du mémoire sont donc multiples. Il s'agira d'approfondir les mécanismes de recherche vectorielle avancée dans Oracle 23ai, en s'appuyant sur des jeux de données plus réalistes et sur des cas d'usage concrets comme la recherche sémantique ou la recommandation. Une évaluation comparative avec d'autres solutions vectorielles open-source est également prévue, afin de mesurer la pertinence et les performances d'Oracle face à des moteurs spécialisés. Enfin, des scénarios d'intégration dans un environnement de data lake, via Hadoop ou Hive, pourront être explorés pour tester la viabilité d'un backend Oracle dans des architectures big data hybrides.

En résumé, cette première étape a permis de mettre en place un socle technique solide pour manipuler des données vectorielles avec Oracle, tout en révélant les pistes à explorer pour tirer pleinement parti de ses capacités dans des environnements modernes orientés IA et traitement sémantique.

8. Intégration d'oracle 23ai et les base de données vectoriels dans un environnement Bigdata/data lake

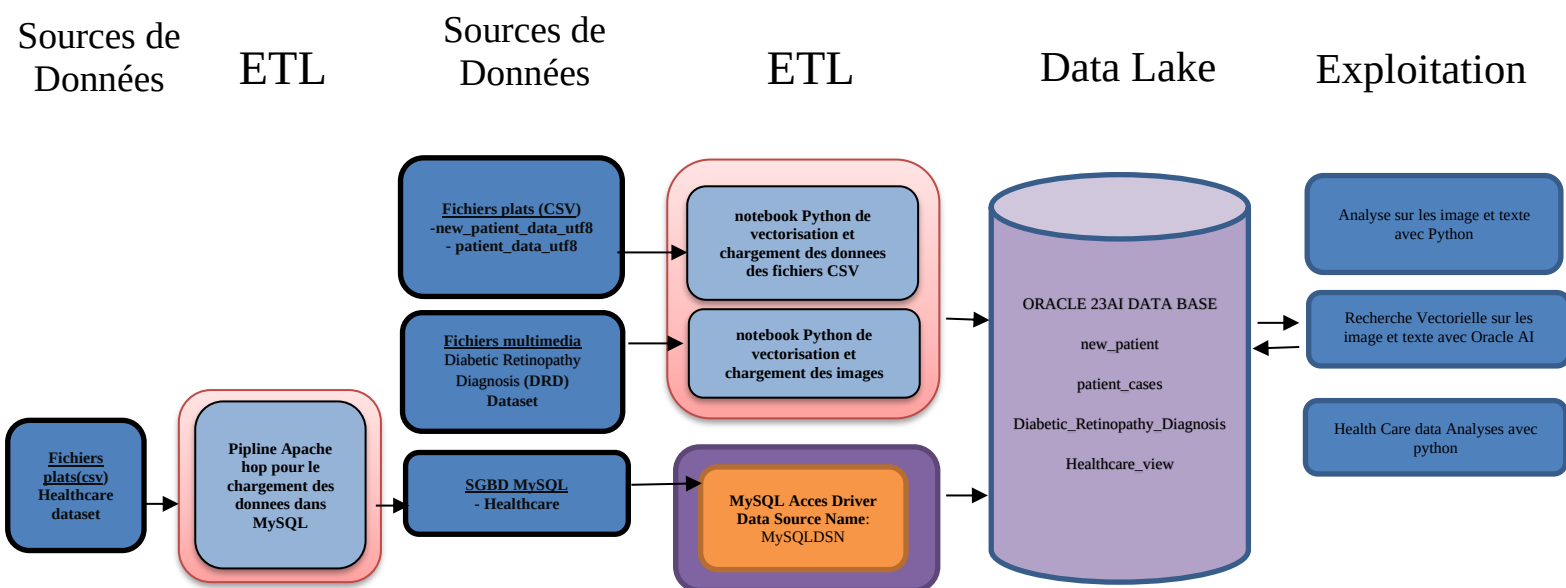
8.1 Introduction

L'explosion des données hétérogènes (textes, images, signaux, vidéos, etc.) dans le domaine de la santé exige des solutions capables de gérer à la fois des données relationnelles traditionnelles et des données non structurées. Dans ce contexte, **Oracle Database 23AI** s'impose comme une plateforme hybride permettant l'intégration de données vectorielles directement dans le moteur relationnel. Cette capacité permet non seulement d'exploiter les données transactionnelles, mais aussi de réaliser des analyses avancées basées sur l'intelligence artificielle (IA) et le Machine Learning.

L'objectif de ce chapitre est de présenter l'intégration d'Oracle 23AI avec les bases de données vectorielles dans un **environnement Big Data/Data Lake**, en s'appuyant sur l'architecture construite dans ce projet.

8.2 Architecture générale du Data Lake

L'architecture mise en place repose sur une approche hybride, combinant des sources de données multiples (CSV, images médicales, SGBD MySQL) et leur intégration dans un **Data Lake centré sur Oracle 23AI**.



8.2.1 Sources de données

- **Dataset Healthcare** : Obtenus sur [Kaggle - Healthcare Dataset](#), Cet ensemble de données synthétiques sur la santé a été créé pour servir de ressource précieuse aux passionnés de science des données, d'apprentissage automatique et d'analyse de données. Il comprend 10 000 enregistrements, chacun représentant un dossier médical synthétique d'un patient.
- **Dataset Diabetic Retinopathy Diagnosis (DRD)** : Obtenus sur [Kaggle - Diabetic Retinopathy Diagnosis Dataset](#) constitué d'une vaste collection de 1505 fichiers d'images rétinienne permettant de diagnostiquer la rétinopathie diabétique. Les images rétinienne sont capturées à l'aide d'une caméra de fond d'œil, un appareil spécialisé pour l'imagerie haute résolution de l'intérieur de l'œil. Les images collectées sont anonymisées et organisées pour supprimer les informations personnelles, garantissant

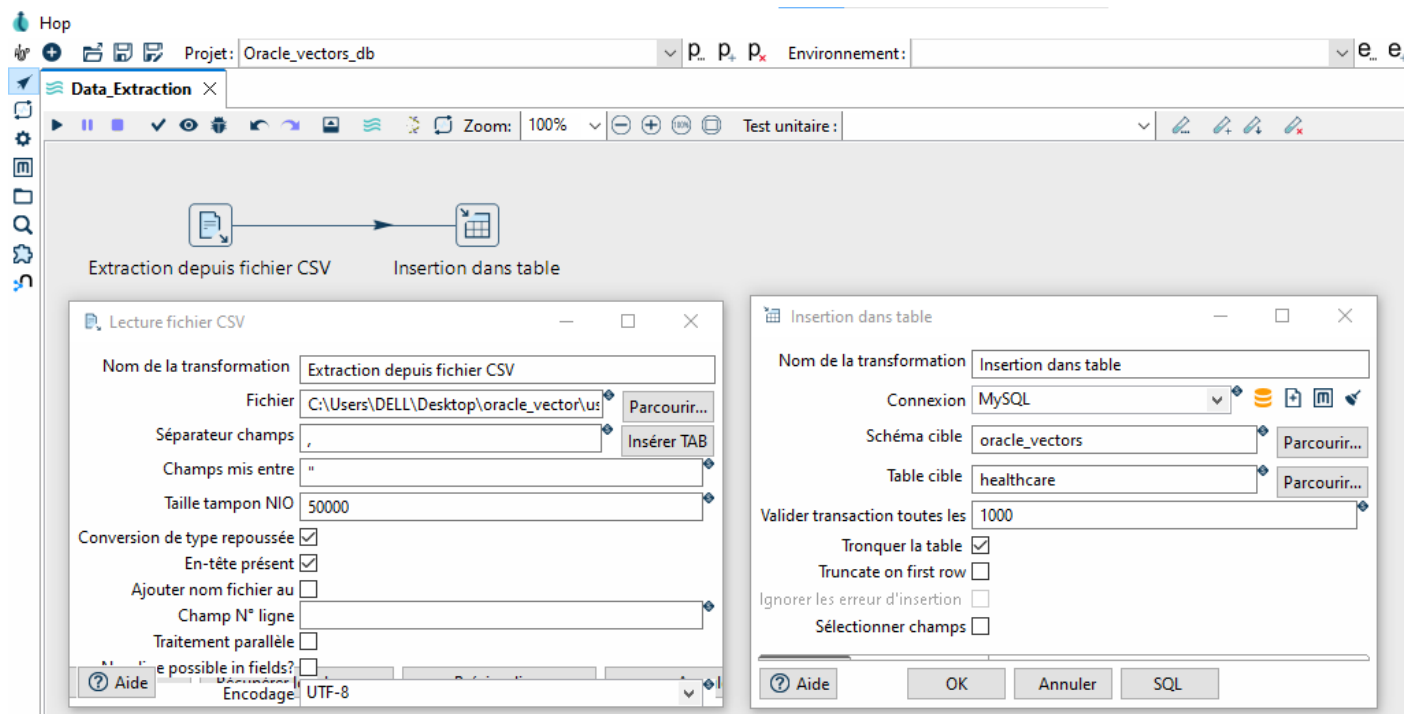
ainsi la confidentialité des patients. L'objectif ultime est d'améliorer les résultats des patients et les stratégies de traitement de la rétinopathie diabétique.

- **Fichiers CSV générés via IA (ChatGPT)** : simulant des données pour la recommandation de traitement médical à l'aide de bases de données vectorielles. L'objectif ici est de Faciliter la prise de décision médicale en recommandant automatiquement des traitements adaptés à partir de cas médicaux similaires déjà traités, en s'appuyant sur l'intelligence artificielle et les bases vectorielles.

Ces sources illustrent la **variété des données** rencontrées dans les systèmes Big Data : données tabulaires, images médicales, fichiers semi-structurés.

8.2.2 Intégration via MySQL et Apache Hop

Les données tabulaires sont d'abord intégrées dans un **SGBD intermédiaire MySQL**, grâce à des pipelines construits dans **Apache Hop**. Apache Hop permet de gérer les flux ETL (Extract, Transform, Load) de manière dynamique et modulaire, avec une gestion centralisée des connexions aux bases de données (via variables et métadonnées).



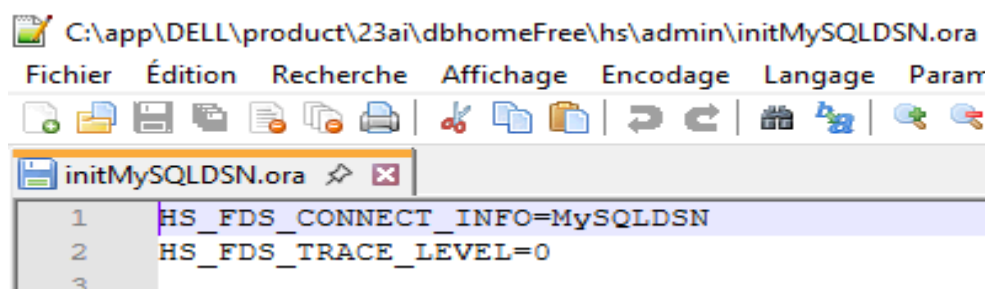
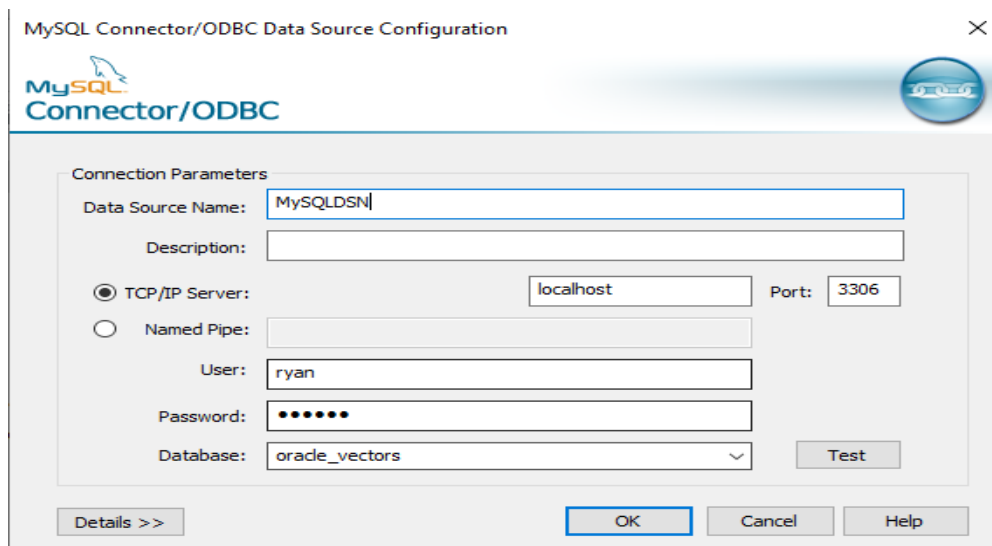
- ❖ Une connexion est établie avec MySQL en local via XAMPP et une Table est créé pour accueillir les données d'Apache hop

```
<MySQL> Creation MySQL tables.sql X
-- Création de la table healthcare avec une clé primaire
CREATE TABLE healthcare (
  patient_id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255),
  age INT,
  gender VARCHAR(6),
  blood_type VARCHAR(3),
  medical_condition VARCHAR(12),
  date_of_admission DATE,
  doctor VARCHAR(255),
  hospital VARCHAR(255),
  insurance_provider VARCHAR(16),
  billing_amount INT,
  room_number INT,
  admission_type VARCHAR(9),
  discharge_date DATE,
  medication VARCHAR(11),
  test_results VARCHAR(12)
);
```

8.2.3 Passerelle ODBC entre Oracle et MySQL

Pour intégrer ces données dans Oracle, nous avons configuré une **gateway hétérogène (dg4odbc)** permettant d'établir un **database link** entre Oracle 23AI et MySQL.

- Le pilote [MySQL ODBC \(MySQLDSN\)](#) joue ici le rôle de connecteur.
- Cela permet à Oracle de requêter directement les données stockées dans MySQL, comme si elles faisaient partie de sa base interne.



8.2.4 Stockage centralisé dans Oracle 23AI

Toutes les données sont finalement intégrées et stockées dans **Oracle Database 23AI**, qui devient le socle du Data Lake.

- Les données tabulaires sont intégrées dans des tables Oracle via des vues externes et des tables de staging.
- Les images médicales sont vectorisées via des scripts Python (Sentence Transformers pour le texte, modèles de vision pour les images).

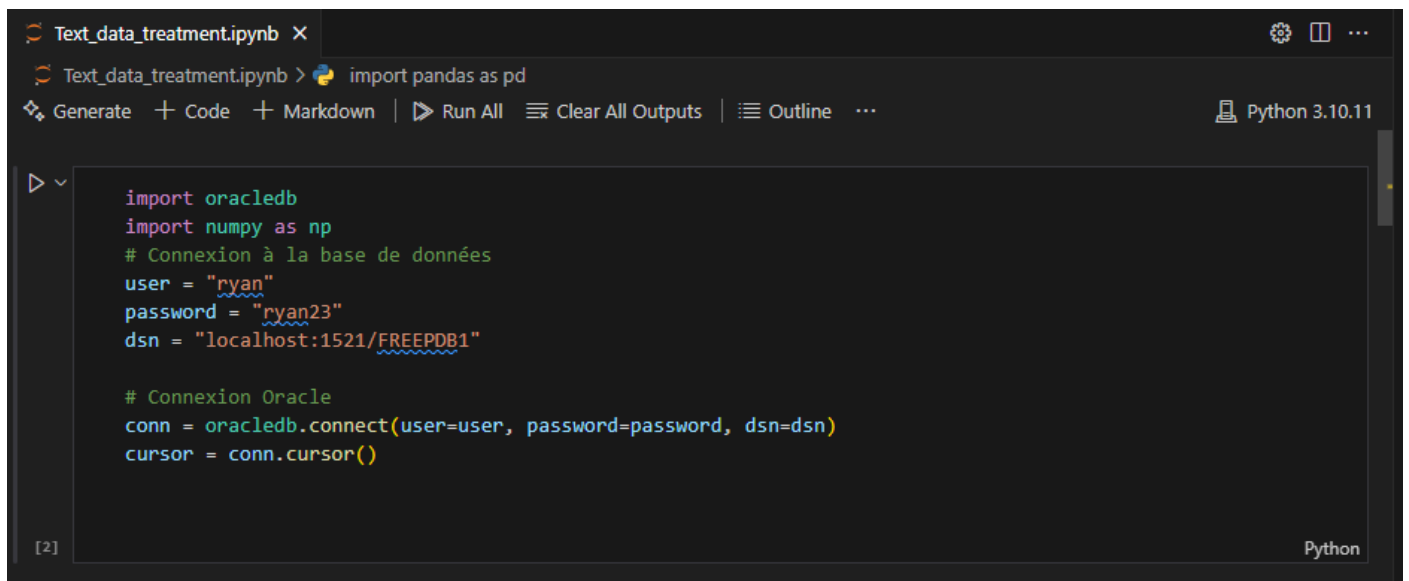
8.3 Oracle 23AI comme moteur vectoriel dans le Data Lake

L'innovation majeure de la version 23AI d'Oracle est son **moteur vectoriel intégré**, qui permet de :

1. Stocker des **vecteurs denses** (représentations numériques des textes et images).
2. Réaliser des recherches de **similarité sémantique** directement en SQL.
3. Combiner les vecteurs avec des données relationnelles pour des cas d'usage concrets.

8.3.1 Génération des vecteurs

1. Pour le texte (fichiers CSV, dossiers patients), nous avons utilisé le modèle **BioBERT-mnli-snli-scinli-scitail-mednli-stsb** de Sentence Transformers.



```
Text_data_treatment.ipynb X
Text_data_treatment.ipynb > import pandas as pd
Generate + Code + Markdown | Run All Clear All Outputs | Outline ... Python 3.10.11

import oracledb
import numpy as np
# Connexion à la base de données
user = "ryan"
password = "ryan23"
dsn = "localhost:1521/FREEPDB1"

# Connexion Oracle
conn = oracledb.connect(user=user, password=password, dsn=dsn)
cursor = conn.cursor()
```

Text_data_treatment.ipynb ×

Text_data_treatment.ipynb > import pandas as pd

Generate + Code + Markdown | Run All Clear All Outputs | Outline ... Python 3.10.11

```
# Charger les données CSV
df = pd.read_csv("patient_data_utf8.csv", sep = ";")
df.head()
```

[3] Python

	# ID	Patient Description	Treatment	Outcome
0	1	45-year-old patient presenting with	Inhaled corticosteroids and broncho	Success
1	2	32-year-old woman suffering from c	Beta blockers and relaxation session	Partial
2	3	60-year-old man with type 2 diabete	Oral antibiotics and local care	Success
3	4	50-year-old patient with exertional c	Statins and cardiac rehabilitation pro	Success
4	5	28-year-old woman presenting with	Antibiotics (fosfomycin) and abunda	Success

5 rows x 4 cols 10 per page << < Page 1 of 1 > >>

```
import pandas as pd
from sentence_transformers import SentenceTransformer

# Charger BioBERT fine-tuné
model = SentenceTransformer("pritamdeka/BioBERT-mnli-snli-scinli-scitail-mednli-stsb")
```

[1] Python

```
# Encodage et insertion
for _, row in df.iterrows():
    vector = model.encode(row["Patient Description"])
    # Normalisation L2
    vector = vector / np.linalg.norm(vector)
    vector_str = "[" + ",".join(map(str, vector)) + "]"

    cursor.execute("""
        INSERT INTO patient_cases (id, description, traitement_prescrit, outcome, vector)
        VALUES (:1, :2, :3, :4, TO_VECTOR(:5))
    """, (int(row["ID"]), row["Patient Description"], row["Treatment"], row["Outcome"], vector_str))

conn.commit()
cursor.close()
conn.close()
print("Insertion terminée avec succès !")
```

[4] Python

... Insertion terminée avec succès !

2. Pour les images médicales (DRD), nous avons utilisé le modèle de vision **CLIP de openAI** afin d'extraire des embeddings vectoriels.

Insertion d'un fichier d'image

```
import os
import numpy as np
from PIL import Image
from transformers import CLIPProcessor, CLIPModel
import oracledb

# -----
# 1. Connexion Oracle
# -----
conn = oracledb.connect(
    user="ryan",
    password="ryan23",
    dsn="localhost:1521/FREEPDB1"
)
cur = conn.cursor()

# -----
# 2. Charger le modèle CLIP
# -----
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
```

[10]

Python

```
# -----
# 3. Encoder une image → vecteur
# -----
def encode_image(image_path):
    image = Image.open(image_path).convert("RGB")
    inputs = processor(images=image, return_tensors="pt")
    with torch.no_grad():
        embeddings = model.get_image_features(**inputs)
    # Normalisation du vecteur
    vector = embeddings[0].numpy()
    vector = vector / np.linalg.norm(vector)
    return vector.tolist() # convertir en liste Python
```

[3]

Python

Generate

+ Code

+ Markdown

```

# -----
# 4. Parcourir le dossier d'images
# -----
folder_path = "DRD Dataset" # dossier contenant tes images
for filename in os.listdir(folder_path):
    if filename.lower().endswith((".jpg", ".jpeg", ".png")):
        img_path = os.path.join(folder_path, filename)
        try:
            # Encoder en vecteur
            vec = encode_image(img_path)
            vector_str = "[" + ",".join(map(str, vec)) + "]"

            # Lire l'image en binaire
            with open(img_path, "rb") as f:
                img_blob = f.read()

            # Insertion dans Oracle
            cur.execute("""
                INSERT INTO Diabetic_Retinopathy_Diagnosis (image_name, embedding, image_data)
                VALUES (:1, TO_VECTOR(:2), :3)
            """, (filename, vector_str, img_blob))

            #print("image + vecteur insérés avec succès")
        except Exception as e:
            print(f"[ERREUR] {filename} : {e}")

conn.commit()
cur.close()
conn.close()

```

8.3.2 Stockage des vecteurs dans Oracle

Les vecteurs générés sont stockés dans des tables Oracle dédiée, par exemple :

```

CREATE TABLE patient_cases (
    id NUMBER PRIMARY KEY,
    description CLOB,
    traitement_prescrit VARCHAR2(500),
    outcome VARCHAR2(100),
    vector VECTOR(768, FLOAT32)
);

desc patient_cases;

CREATE TABLE new_patient (
    id NUMBER PRIMARY KEY,
    description CLOB,
    vector VECTOR(768, FLOAT32)
);

desc new_patient;

```

De même, une table parallèle est définie pour les vecteurs d'images.

```

CREATE TABLE DIABETIC_RETINOPATHY_DIAGNOSIS (
    image_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    image_name VARCHAR2(255),
    embedding VECTOR(512),           -- vecteur d'encodage
    image_data BLOB                  -- stockage de l'image brute
);

```

8.4 Analyses avancées dans le Data Lake

8.4.1 Analyses PCA et DBSCAN sur le texte et les images des données médicales

L'intégration d'Oracle 23AI permet de réaliser des analyses avancées telles que :

- PCA pour visualiser la distribution des vecteurs.
- Regroupement de patients par similarité vectorielle (groupes thématiques) avec DBSCAN.

8.4.1.1 PCA pour visualiser la distribution des vecteurs

Les Données sont récupérées depuis la base de données Oracle et traiter avec python

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
import oracledb
import json
import array

# Connexion Oracle
conn = oracledb.connect(user="ryan", password="ryan23", dsn="localhost:1521/FREEDB1")
cursor = conn.cursor()
# Récupération des vecteurs
cursor.execute("SELECT id, description, vector FROM patient_cases")
data = cursor.fetchall()

vectors = []
labels = []

for row in data:
    vec = row[2] # colonne vector
    try:
        if isinstance(vec, oracledb.LOB): # --- Si c'est un LOB (CLOB/BLOB) ---
            vec = vec.read()
        if isinstance(vec, str): # --- Si c'est une chaîne JSON ---
            vec = json.loads(vec)
        elif isinstance(vec, array.array): # --- Si c'est un array.array ---
            vec = list(vec)
        elif isinstance(vec, list): # --- Si c'est déjà une liste ---
            pass
        else: # --- Sinon format inconnu ---
            print(f" Erreur sur ID {row[0]}: format inattendu ({type(vec)})")
            continue

        # Vérification du contenu
        if isinstance(vec, list) and len(vec) > 0:
            vectors.append(vec)
            labels.append(f"ID {row[0]}")
            clob_content = row[1].read() # lire le contenu du CLOB description
            description_preview = (clob_content[:30] + '...') if len(clob_content) > 30 else clob_content
        else:
            print(f" Erreur sur ID {row[0]}: vecteur vide ou invalide")
    except Exception as e:
        print(f" Erreur sur ID {row[0]}: {e}")

print(f" {len(vectors)} vecteurs récupérés")
```

✓ 0.1s

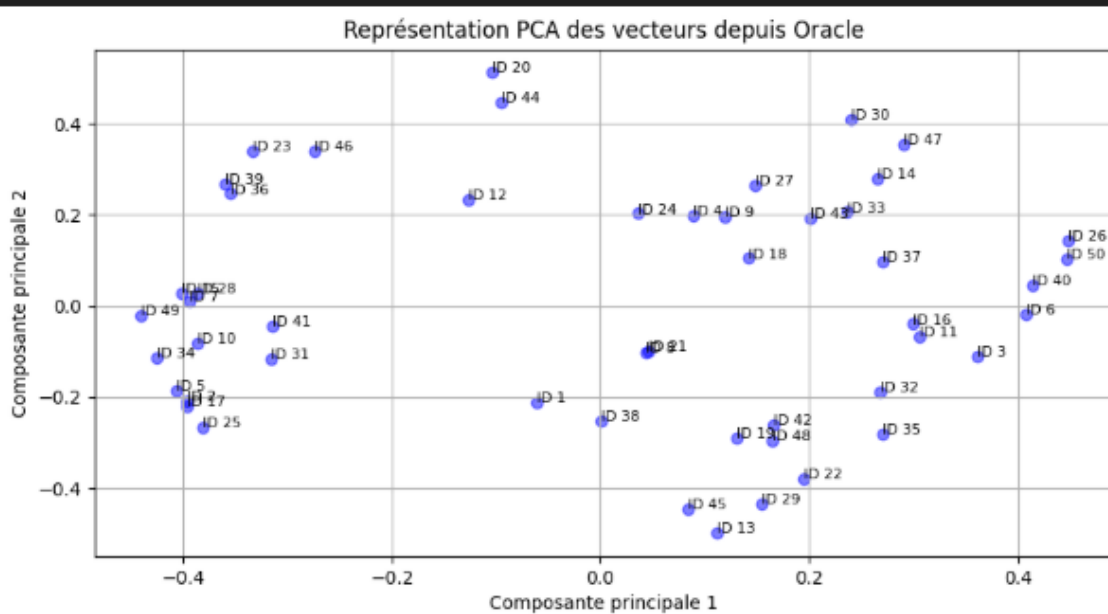
50 vecteurs récupérés

L'analyse PCA ici nous montre les résultats suivants :

```
# Représentation PCA si vecteurs valides
if vectors:
    X = np.array(vectors)
    pca = PCA(n_components=2)
    reduced = pca.fit_transform(X)

    plt.figure(figsize=(10, 5))
    plt.scatter(reduced[:, 0], reduced[:, 1], marker='o', color='blue', alpha=0.5)
    for i, label in enumerate(labels):
        plt.annotate(label, (reduced[i, 0], reduced[i, 1]), fontsize=8)
    plt.title("Représentation PCA des vecteurs depuis Oracle")
    plt.xlabel("Composante principale 1")
    plt.ylabel("Composante principale 2")
    plt.grid(True)
    plt.show()
else:
    print("Aucun vecteur valide à afficher.")
```

✓ 0.9s



□ Axes principaux :

- L'axe X (**Composante principale 1**) et l'axe Y (**Composante principale 2**) représentent les **deux directions de variance maximale** dans tes données.
- Autrement dit, ces axes concentrent le maximum d'information des vecteurs d'origine (souvent des centaines de dimensions).

□ Répartition des points :

- Les points **proches les uns des autres** représentent des **vecteurs (patients) ayant des diagnostics très similaires**
- Les points **éloignés** traduisent **des patients avec des cas particuliers**.

□ Groupes visibles :

- On observe plusieurs **zones de regroupement** (clusters naturels) :
 - Par exemple, autour de la zone **(-0.4, -0.2)** et **(0.3, -0.4)**.
 - Cela suggère qu'il existe des sous-groupes de vecteurs (patients) ayant des diagnostics proches.
- Les points isolés comme **ID 1, 20 ou 91** peuvent être patients avec des diagnostics à part

□ **Valeurs centrées :**

- Les vecteurs sont bien centrés autour de (0,0), ce qui indique que le PCA a correctement normalisé les composantes principales.

8.4.1.2 Regroupement de patients par similarité vectorielle (groupes thématiques) avec DBSCAN.

DBSCAN sur les données de text

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

# On standardise les données (important pour DBSCAN)
X_scaled = StandardScaler().fit_transform(X)
```

✓ 0.0s

```
# DBSCAN avec des paramètres initiaux
dbscan = DBSCAN(eps=0.8, min_samples=2)
db_labels = dbscan.fit_predict(X)
```

✓ 0.0s

```
plt.figure(figsize=(10, 4))
# Affichage des points avec couleur selon leur cluster
unique_labels = np.unique(db_labels)
colors = plt.cm.get_cmap("tab10", len(unique_labels))

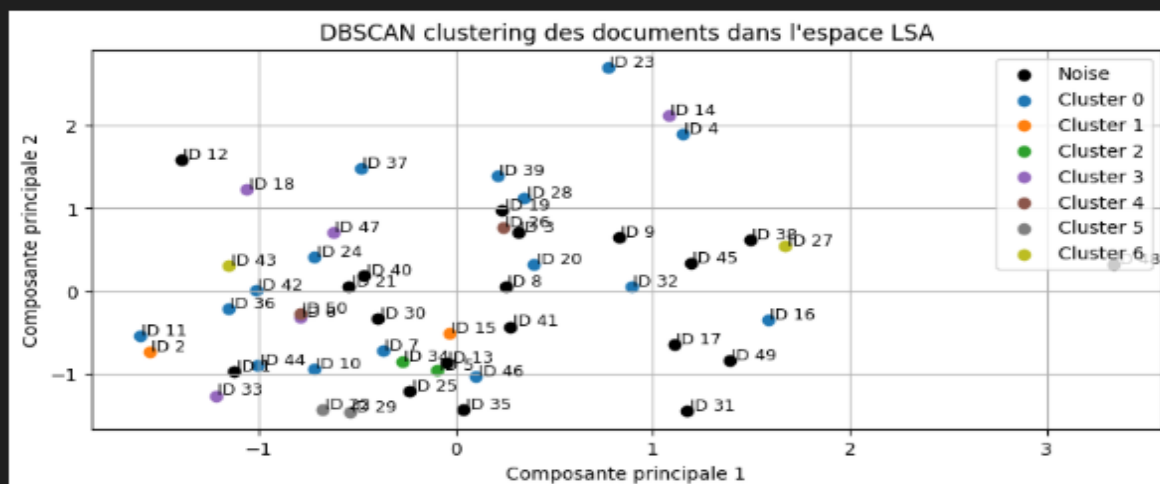
for label in unique_labels:
    mask = db_labels == label
    color = 'black' if label == -1 else colors[label]
    plt.scatter(X_scaled[mask, 0], X_scaled[mask, 1], label=f"Cluster {label}" if label != -1 else "Noise", c=[color])

# Affichage des étiquettes
for i, txt in enumerate(labels[:500]):
    plt.annotate(txt, (X_scaled[i, 0]+0.01, X_scaled[i, 1]+0.01), fontsize=9)

plt.title("DBSCAN clustering des documents dans l'espace LSA")
plt.xlabel("Composante principale 1")
plt.ylabel("Composante principale 2")
plt.legend()
plt.grid(True)
plt.show()
```

✓ 1.3s

```
C:\Program Files\KMSpico\temp\ipykernel_21856\3887556289.py:7: MatplotlibDeprecationWarning: The get_cmap function was deprecated
colors = plt.cm.get_cmap("tab10", len(unique_labels))
```



Interprétation du clustering DBSCAN dans l'espace LSA

L'analyse présentée ci-dessus illustre les résultats de l'algorithme **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** appliqué aux vecteurs de patients projetés dans un espace **LSA (Latent Semantic Analysis)** à deux dimensions à l'aide d'une réduction par **PCA**.

1. Structure générale de la répartition

Le graphique montre une distribution des documents selon deux axes principaux :

- **Composante principale 1**, représentant la première direction de variance maximale ;
- **Composante principale 2**, correspondant à la seconde direction de variance.

Chaque point du graphique correspond au vecteur d'un patient, et chaque **couleur distincte** représente un **cluster identifié par DBSCAN**. Les **points noirs**, quant à eux, sont étiquetés comme « **Noise** » (bruit), c'est-à-dire des éléments que l'algorithme n'a pas réussi à regrouper dans un cluster en raison d'une densité trop faible autour d'eux.

2. Résultats du clustering

L'exécution de DBSCAN a permis d'identifier **sept clusters distincts (Cluster 0 à Cluster 6)** ainsi qu'un ensemble de points considérés comme du bruit. Ces clusters correspondent à des **zones de forte densité** dans l'espace vectoriel, traduisant des **similarités** entre les patients. En d'autres termes, les patients appartenant à un même cluster partagent des caractéristiques proches.

La présence de plusieurs groupes bien délimités indique que les vecteurs produits par le modèle de représentation (LSA) sont suffisamment discriminants pour permettre une séparation naturelle des patients selon leur diagnostic.

3. Interprétation des points isolés (« Noise »)

Les points classés comme « Noise » représentent des **patients atypiques**, possiblement :

- Des patients abordant des cas marginaux par rapport au reste de patients ;
- Ou des cas où la densité locale n'était pas suffisante pour satisfaire les paramètres de DBSCAN (eps et min_samples).

Leur proportion modérée dans le graphique montre néanmoins un **bon équilibre** entre la détection de clusters et la gestion du bruit.

4. Conclusion

Cette représentation met en évidence la **capacité de DBSCAN à détecter automatiquement des groupes de patients similaires** sans connaissance préalable du nombre de catégories. La présence de sept clusters distincts, combinée à un faible niveau de bruit, confirme la pertinence du modèle de vectorisation et la bonne qualité de la réduction dimensionnelle appliquée.

```

description_previews = []
for row in data: # rows = liste des résultats de la DB
    clob_content = row[1].read() if hasattr(row[1], 'read') else row[1]
    preview = (clob_content[:200] + '...') if len(clob_content) > 200 else clob_content
    description_previews.append(preview)

```

✓ 0.0s

```

for label in unique_labels:
    print(f"\n Cluster {label if label != -1 else 'Noise'}:")
    for i in range(len(labels[:500])):
        if db_labels[i] == label:
            print(f" - Doc{i+1}: {description_previews[i]}")

```

✓ 0.0s

- Doc46: 44-year-old man with post-traumatic stress disorder.
 - Doc48: 62-year-old woman with knee osteoarthritis.
 - Doc50: 53-year-old woman with iron deficiency anemia.

Cluster 1:

- Doc4: 32-year-old woman suffering from chronic migraines triggered by stress and lack of sleep.
 - Doc25: 25-year-old woman with generalized anxiety and sleep disorders.

Cluster 2:

- Doc7: 28-year-old woman presenting with acute urinary tract infection with burning during urination.
 - Doc32: 27-year-old woman with recurrent cystitis.

Cluster 3:

- Doc8: 72-year-old man with chronic respiratory failure worsened by viral infection.
 - Doc12: 80-year-old patient suffering severe dehydration after gastroenteritis.
 - Doc24: 70-year-old patient with Parkinson's disease, increased muscle rigidity.
 - Doc31: 74-year-old patient with chronic bronchitis.
 - Doc41: 78-year-old patient with heart failure.

Cluster 4:

- Doc16: 60-year-old man with atrial fibrillation.
 - Doc44: 61-year-old man with recent ischemic stroke.

Cluster 5:

- Doc18: 37-year-old man with poorly controlled asthma.
 - Doc39: 39-year-old man with severe influenza.

Cluster 6:

- Doc37: 55-year-old patient with stage 3 chronic kidney disease.
 - Doc47: 59-year-old patient with acute pancreatitis.

8.4.2 Recherche de Similarité Vectorielle sur les image et texte en SQL avec Oracle AI

8.4.3 Health Care data Analyses avec python

8.5 Apport de l'intégration Oracle 23AI dans un Data Lake

1. **Centralisation** : toutes les données (tabulaires, CSV, images vectorisées) sont intégrées dans un même socle Oracle.
2. **Hybridation** : grâce à la gateway ODBC, Oracle devient le point d'accès unique, même pour des données provenant de MySQL.
3. **Performance** : Oracle 23AI optimise le stockage et la recherche vectorielle nativement.
4. **Interopérabilité** : possibilité de croiser recherche vectorielle et données relationnelles en SQL.
5. **Exploitabilité** : analyses avancées possibles via Python et outils BI.

8.6 Conclusion

Ce chapitre a montré comment l'intégration d'Oracle 23AI dans un environnement Big Data/Data Lake permet de combiner le meilleur des deux mondes : la robustesse des bases de données relationnelles et la puissance des bases vectorielles. Grâce à l'architecture mise en place, il est possible de réaliser des cas d'usage concrets en santé : recherche sémantique, comparaison d'images médicales, détection d'anomalies, ou encore analyses prédictives.

9. Conclusion générale et perspectives

Principaux apports de l'étude pour vous, pour l'entreprise ou celui qui a défini le sujet, pour l'école
Précisions sur le travail réalisé (état de l'art, installation, étude ORACLE 23AI ET LES BD VECTORIELS, mise en œuvre du cas d'usage, intégration dans l'environnement Bigdata, ...)
Précisions sur la situation finale du travail réalisée (Le statu des livrables en production, ..., en test, fini à X %)
Principales difficultés rencontrées et solutions apportées
Perspectives ou encore ce qui reste à faire ou encore les extensions futures

10. Bibliographie

Chapitre 3 – Concepts fondamentaux des bases de données vectorielles

1. **Johnson, J., Douze, M., & Jégou, H. (2019).** *Billion-scale similarity search with GPUs*. IEEE Transactions on Big Data, 7(3), 535–547.
- Référence pour la **recherche vectorielle, ANN, brute force vs approximate**.
2. **Wang, J., Zhang, T., Song, J., Sebe, N., & Shen, H. T. (2018).** *A survey on learning to hash*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 40(4), 769–790.
- Appuie la partie **méthodes d'indexation (hashing, ANN)**.
3. **Manning, C., Raghavan, P., & Schütze, H. (2008).** *Introduction to Information Retrieval*. Cambridge University Press.
- Base théorique sur les **mesures de similarité (cosine, Euclidean, Jaccard)**.

Chapitre 4 – État de l'art et critères comparatifs

Fondements théoriques et architecture

- **Oracle 23ai**
 - Oracle (2024). *Oracle Database 23ai: The AI Vector Database*. [Oracle Vector Database](#)
 - Oracle (2024). *Introduction to Oracle AI Vector Search*.
- **PostgreSQL + pgvector**
 - Zilliz (2023). *Postgres + pgvector for AI*. [pgvector GitHub](#)
 - Feng, C. (2023). *Vector Similarity Search in PostgreSQL with pgvector*. *Towards Data Science*.
- **Weaviate**
 - Weaviate (2024). *Weaviate Documentation*.
 - Chia, C. (2023). *A Deep Dive into Weaviate: An Open-Source Vector Database*. *Medium*.

- **FAISS**

- Johnson, J., Douze, M., & Jégou, H. (2017). *Billion-scale similarity search with GPUs*. *IEEE Transactions on Big Data*.
- Facebook AI Research (2023). *FAISS Library*. [GitHub](#)

Chapitre 5 – Oracle 23ai et BD vectorielles

8. **Oracle. (2024).** *Oracle Database 23ai New Features Guide*. Oracle Documentation.
- Référence directe pour **Oracle 23ai et la gestion native des vecteurs**.
9. **Oracle. (2024).** *AI Vector Search with Oracle Database 23ai*. White Paper.
- Document clé pour la section **application de référence et intégration AI + BD vectorielles**.
10. **Oracle Blogs. (2024).** *Oracle Database 23ai: Native Vector Search for AI*.
- Appuie la **présentation des objectifs d'Oracle 23ai**.

Chapitre 6 – Intégration de ORACLE 23AI ET LES BD VECTORIELS dans un environnement Bigdata/data lake

Documentation officielle Oracle

- Oracle Corporation. (2024). *Oracle Database 23ai: New Features Guide*. Oracle Help Center.
Disponible sur : <https://docs.oracle.com/en/database/oracle/oracle-database/23/>
- Oracle Corporation. (2024). *Database Gateway for ODBC User's Guide*. Oracle Help Center.
Disponible sur : <https://docs.oracle.com/en/database/oracle/oracle-database/23/odbcu/>
- Oracle Corporation. (2024). *Error Messages Reference*. Oracle Help Center.
Disponible sur : <https://docs.oracle.com/error-help/db/>

Documentation MySQL et ODBC

- Oracle Corporation. (2024). *MySQL ODBC Connector Documentation (MyODBC)*.
Disponible sur : <https://dev.mysql.com/doc/connector-odbc/en/>
- MariaDB Foundation. (2025). *MariaDB Server Documentation*.
Disponible sur : <https://mariadb.com/kb/en/documentation/>

Apache Hop (ETL / Data Integration)

- The Apache Software Foundation. (2025). *Apache Hop Documentation*.
Disponible sur : <https://hop.apache.org/docs/>

NLP et Vectorisation

- Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. arXiv:1908.10084.
Disponible sur : <https://arxiv.org/abs/1908.10084>
- Hugging Face. (2025). *Sentence-Transformers Documentation*.
Disponible sur : <https://www.sbert.net>

Bases de données vectorielles et IA

- Milvus. (2025). *Milvus Open-Source Vector Database*.
Disponible sur : <https://milvus.io>
- Facebook AI Research. (2017). *FAISS: A library for efficient similarity search and clustering of dense vectors*.
Disponible sur : <https://faiss.ai>

Jeux de données (sources Kaggle)

- Kaggle. (2025). *Healthcare Dataset*.
Disponible sur : <https://www.kaggle.com/> (recherche : "Healthcare dataset")
- Kaggle. (2025). *Diabetic Retinopathy Detection Dataset*.
Disponible sur : <https://www.kaggle.com/> (recherche : "Diabetic Retinopathy Detection")

Big Data et Data Lakes

- Marz, N., & Warren, J. (2015). *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications.
- Singh, M., & Reddy, C. K. (2014). *A survey on platforms for big data analytics*. Journal of Big Data, 1(1), 8.